gates 매뉴얼

도구형 Windows 프로그램을 위한 작은 C23 retained GUI 라이브러리

판 0.15.1

https://github.com/rubidus-api/gates_gui_lib

영어판이 원본이고 이 판은 그 번역이다. 둘이 어긋나면 영어판을 따른다.

MIT License.

차례

0. 여기서부터 1
  0.1 gates 는 무엇을 위한 것인가 1
  0.2 패키지 1
  0.3 첫 프로그램 1
  0.4 창 없이 3
  0.5 헤더 3

1. 개념 5
  1.1 노드 트리 5
  1.2 소유: 복사하는 것과 빌리는 것 6
  1.3 오류 6
  1.4 이벤트와 안전한 시점 6
  1.5 단위 7

2. 컨트롤과 배치 8
  2.1 컨트롤 표 8
  2.2 배치 10

3. 폼 12
  3.1 폼은 데이터를 갖지 않는다 12
  3.2 초안 검사 13

4. 명령, 포커스, 대화상자, 메뉴 14
  4.1 동작 하나, 청하는 길 여럿 14
  4.2 포커스 15
  4.3 대화상자 15
  4.4 문맥 메뉴 15
  4.5 실행 취소와 다시 실행 15

5. 내 데이터 위의 뷰 18
  5.1 줄이 몇 개든 노드 하나 18
  5.2 목록, 표, 트리, 로그 19
  5.3 사람이 바꿀 수 있는 칸 20
  5.4 여러 줄을 한꺼번에: 다중 선택 22
  5.5 한 레코드의 필드: 속성 격자 24
  5.6 모델의 규칙 25

6. 글자 입력 26
  6.1 텍스트 상자 26
  6.2 한도는 질문이다 26
  6.3 설치된 입력기 27
  6.4 0.11.0 의 유니코드 한계 27
  6.5 클립보드 27
  6.6 바깥의 다른 인코딩 28
  6.7 여러 줄: 편집기 29
  6.8 큰 글 32

7. 작업 스레드와 타이머 33
  7.1 작업 스레드는 트리를 만지지 않는다 33
  7.2 백그라운드 작업 35
  7.3 타이머 36

8. 테마, 단위, 배율 38
  8.1 논리 단위 38
  8.2 테마 38
  8.3 글꼴 39

8.4 그리기와 글자 백엔드 39
9. 접근성 40
9.1 모든 창은 눈과 포인터 없이도 읽고 쓸 수 있다 40
9.2 이름 40
9.3 강제하는 규칙 41
9.4 소리 내어 알리기 41
9.5 내레이터로 확인하기 41
10. 배포와 문제 해결 42
10.1 링크 42
10.2 판과 호환 42
10.3 응용과 창 42
10.4 문제 해결 43
11. 응용 프로그램 틀 44
11.1 니모닉 44
11.2 메뉴 막대 44
11.3 키맵: 단축키 나열과 다시 묶기 45
11.4 도구 막대 46
11.5 상태 줄 46
11.6 툴팁 47
11.7 탭 48
11.8 배치 기억하기 48
11.9 접근성 50
12. 숫자, 묶음, 배치 51
12.1 여러 컨트롤에 처리기 하나 51
12.2 한 차례에 한 번: 미룬 호출 51
12.3 스핀 상자와 슬라이더 51
12.4 그룹 상자 53
12.5 격자와 줄바꿈 배치 53
12.6 접근성 54
13. 이미지와 네이티브 대화 상자 55
13.1 이미지 55
13.2 아이콘 55
13.3 네이티브 대화 상자 56

0 여기서부터

이 장의 차례

0.1 gates 는 무엇을 위한 것인가 1
0.2 패키지 1
0.3 첫 프로그램 1
0.4 창 없이 3
0.5 헤더 3

0.1 gates 는 무엇을 위한 것인가

gates 는 사람이 무언가를 보고 바꾸게 하는 것이 일인 프로그램을 위한 GUI 라이브러리다. 설정 창, 기록 살펴보기, 로그 보기, 빌드 도구가 그런 프로그램이다. C23 으로 쓰였고, 화면을 설명하는 노드 트리를 유지하며, 이런 프로그램이 저마다 서툴게 다시 만들던 일을 맡는다. 키보드 포커스, 설치된 입력기로 하는 글자 입력, 폼, 큰 데이터 위의 목록, 버튼과 단축키가 함께 쓰는 명령, 대화상자, 화면에 결과를 넘기는 스레드, 테마, 배율, 접근성이 그 일이다.

브라우저 엔진도, 게임 UI 도, 픽셀까지 똑같이 그리는 도구도 아니다. HTML 과 CSS 를 떠올리면 된다. 프로그램은 무엇이 무엇이며 서로 어떤 관계인지를 말하고, 마지막 모습은 백엔드와 테마가 정한다. gates 에서 기능이 끝났다는 것은 그 기능이 섬기는 상호작용이 동작한다는 뜻이다. 닿을 수 있고, 포인터와 키보드와 입력기로 쓸 수 있고, 화면 읽기 프로그램이 읽을 수 있어야 한다. 그림과 같아 보이는 것은 기준이 아니다.

0.15.1 의 백엔드는 하나, 소프트웨어 그리기를 쓰는 Win32 다. 창을 뺀 나머지 전부인 코어는 플랫폼에 묶이지 않아서 C23 컴파일러가 있는 곳이면 어디서나 돌고, 시험도 그렇게 돈다.

0.2 패키지

릴리스는 `gates-0.15.1/` 폴더 하나다.

경로	무엇
`include/gates/`	공개 헤더. `gates/gates.h` 가 전부를 포함한다
`include/proven/`	gates 가 기반으로 삼는 proven 라이브러리의 헤더
`lib/win64/libgates.a`	Windows 용 gates(코어 + Win32 백엔드), mingw-w64
`lib/host/libgates_core.a`	플랫폼 없는 코어. 시험과 창 없는 도구용
`lib/*/libproven.a`	기반 라이브러리 proven(할당기, 문자열, 결과)
`manual/`, `manual-ko/`	이 매뉴얼의 영문판과 한국어판
`examples/consumer/`	패키지만으로 빌드하는 프로그램과 그 Makefile

gates 와 proven 은 따로 된 라이브러리다. 둘 다 링크하되 gates 를 먼저 둔다. 이미 proven(0.1.x)을 쓰는 프로그램은 패키지의 것 대신 자기 사본을 링크해도 된다.

0.3 첫 프로그램

이름표 하나와 버튼 하나가 있는 창이다. 버튼을 누르면 이름표가 바뀐다.

```
/* manual example (windows): the first program - a window, a label, a button.
 * Build: see chapter 0 (links -lgates -lproven and the Windows libraries). */
#include <gates/gates.h>
```

```c
#include <stdio.h>

typedef struct hello_t {
    gates_app_t *app;
    gates_tree_t *tree;
    gates_node_t count_label;
    int clicks;
} hello_t;

/* The button tells us it was pressed; we change the label. Nothing is read
 * back from widgets while painting. */
static void on_button(gates_tree_t *tree, const gates_event_t *ev, void *user) {
    hello_t *h = user;
    if (ev->kind != GATES_EVENT_ACTIVATED) return;
    h->clicks++;
    char text[48];
    int n = snprintf(text, sizeof text, "Pressed %d time%s", h->clicks, h->clicks == 1 ? "" : "s");
    (void)gates_widget_set_text(tree, h->count_label, (gates_str_t){ .ptr = (const gates_u8 *)text, .size =
(gates_usize_t)n });
}

static void on_key(gates_window_t *win, const gates_key_event_t *ev, void *user) {
    (void)win;
    hello_t *h = user;
    if (ev->down && ev->key == GATES_KEY_ESCAPE) gates_app_quit(h->app);
}

int main(void) {
    gates_app_t *app = nullptr;
    if (!gates_is_ok(gates_app_create(&(gates_app_desc_t){0}, &app))) return 1;
    hello_t h = { .app = app };
    gates_window_desc_t desc = { .title = GATES_STR("Hello, gates"), .size = { 320, 160 } };
    gates_window_callbacks_t cb = { .on_key = on_key, .user_data = &h };
    gates_window_t *win = nullptr;
    if (!gates_is_ok(gates_window_create(app, &desc, &cb, &win))) {
        gates_app_destroy(app);
        return 1;
    }
    h.tree = gates_window_tree(win);
    gates_node_t root = gates_tree_root(h.tree), button;
    gates_err_t err = gates_layout_set(h.tree, root, GATES_LAYOUT_KIND_COLUMN);
    if (gates_is_ok(err)) err = gates_layout_set_padding(h.tree, root, 12);
    if (gates_is_ok(err)) err = gates_label_create(h.tree, root, GATES_STR("Not pressed yet"), &h.count_label);
    if (gates_is_ok(err)) err = gates_button_create(h.tree, root, GATES_STR("Press me"), nullptr, nullptr,
&button);
    if (gates_is_ok(err)) err = gates_widget_set_handler(h.tree, button, on_button, &h);
    if (gates_is_ok(err)) err = gates_app_run(app); /* returns when the last window closes */
    gates_window_destroy(win);
    gates_app_destroy(app);
    return gates_is_ok(err) ? 0 : 1;
}
```

패키지 폴더에서 mingw-w64(Linux, 또는 Windows 의 MSYS2)로 빌드한다.

```
x86_64-w64-mingw32-gcc -std=c23 -O2 -Iinclude hello.c -Llib/win64 -lgates -lproven \
    -lgdi32 -luser32 -limm32 -ldwmapi -ladvapi32 -luiautomationcore -lole32 -loleaut32 \
    -luuid -mwindows -o hello.exe
```

gates 프로그램의 모양은 이 안의 세 가지가 전부다. 프로그램은 창의 트리에 노드를 만들어 넣는다 (`gates_label_create`, `gates_button_create`). 사람이 한 일은 이벤트로 알고 (`gates_widget_set_handler`), 창이 그리는 동안 위젯을 읽어서 알아내지 않는다. 그리고 `gates_app_run` 이 마지막 창이 닫힐 때까지 루프를 맡는다.

0.4 창 없이

코어에는 창이 필요 없다. 트리를 만들고, 내장 글자 백엔드로 배치하고, 무엇이 어디 있는지 묻는다. 시험과 명령줄 도구가 gates 를 이렇게 쓴다.

```c
/* manual example (host): gates without a window - build, lay out, inspect.
 * expect: Press me: button at 0,16 320x26 */
#include <gates/gates.h>

#include <stdio.h>

int main(void) {
    gates_tree_t *tree = nullptr;
    if (!gates_is_ok(gates_tree_create(&(gates_tree_desc_t){0}, &tree))) return 1;
    gates_node_t root = gates_tree_root(tree), label, button;
    gates_err_t err = gates_layout_set(tree, root, GATES_LAYOUT_KIND_COLUMN);
    if (gates_is_ok(err)) err = gates_label_create(tree, root, GATES_STR("Not pressed yet"), &label);
    if (gates_is_ok(err)) err = gates_button_create(tree, root, GATES_STR("Press me"), nullptr, nullptr, &button);
    /* Layout needs text metrics: the builtin backend works everywhere. */
    if (gates_is_ok(err)) err = gates_layout_run(tree, (gates_size_t){ 320, 160 }, gates_text_backend_builtin());
    gates_access_info_t info;
    if (gates_is_ok(err)) err = gates_access_info(tree, button, 0, &info);
    if (gates_is_ok(err)) {
        printf("%.*s: %s at %d,%d %dx%d\n", (int)info.name.size, (const char *)info.name.ptr,
               info.role == GATES_ROLE_BUTTON ? "button" : "?", info.bounds.x, info.bounds.y, info.bounds.w,
               info.bounds.h);
    }
    gates_tree_destroy(tree);
    return gates_is_ok(err) ? 0 : 1;
}
```

```
cc -std=c23 -Iinclude headless.c -Llib/host -lgates_core -lproven -lm -o headless
```

출력은 `Press me: button at 0,16 320x26` 이다. 세로 배치(column)가 버튼을 이름표 아래에 두고 너비 전체로 늘렸다.

0.5 헤더

`gates/gates.h` 가 전부를 포함한다. 이 매뉴얼에서 만나는 차례로 적으면 다음과 같다.

헤더	담긴 것	장
`gates/types.h`	정수, 문자열(`gates_str_t`), 오류, 할당기	1
`gates/version.h`	판과 `gates_version()`	10
`gates/tree.h`	노드 트리, 핸들, 포커스, 숨긴 노드	1
`gates/event.h`	타입 있는 이벤트와 그 전달	1
`gates/widget.h`	이름표, 버튼, 체크박스, 텍스트 상자, 라디오 그룹, 선택 상자, 진행 막대	2
`gates/layout.h`	가로, 세로, 겹침, 나눔, 스크롤, 폼 배치	2
`gates/geometry.h`	점, 크기, 사각형, 논리 단위	2, 8
`gates/form.h`	이름표·도움말·오류 줄이 있는 폼	3
`gates/command.h`	버튼·단축키·메뉴가 함께 쓰는 명령	4
`gates/overlay.h`	모달 대화상자와 문맥 메뉴	4
`gates/view.h`	내 데이터 위의 목록, 표, 트리, 로그	5
`gates/text_edit.h`	모든 텍스트 상자 밑의 편집 코어	6
`gates/clipboard.h`	클립보드 경계	6

헤더	담긴 것	장
`gates/input.h`	포인터와 키 이벤트	6
`gates/ui.h`	입력 전달, 맞힘 판정(hit test), 트리 그리기	6
`gates/post.h`	작업 스레드에서 보내기	7
`gates/timer.h`	UI 스레드의 타이머	7
`gates/theme.h`	테마: 색 토큰, 밝게, 어둡게, 고대비	8
`gates/draw.h`	그리기 명령 목록	8
`gates/render.h`	소프트웨어 그리기	8
`gates/text.h`	글꼴, 글자 치수, 글자 백엔드	8
`gates/access.h`	접근성: 역할, 이름, 동작, 감사	9
`gates/app.h`	응용과 그 루프	10
`gates/window.h`	창, 테마, 확대, 알림	10

1 개념

이 장의 차례

1.1 노드 트리 5
1.2 소유: 복사하는 것과 빌리는 것 6
1.3 오류 6
1.4 이벤트와 안전한 시점 6
1.5 단위 7

헤더: `gates/tree.h`, `gates/types.h`, `gates/event.h`.

1.1 노드 트리

창에 보이는 것은 모두 창 트리의 노드다. 묶는 패널, 이름표, 버튼, 텍스트 상자, 뷰 전체가 그렇다. 프로그램은 노드를 만들어 부모 밑에 두고 바꾼다. gates 는 노드를 배치하고, 그리고, 입력을 보내고, 보조 기술에 설명한다. 트리는 유지된다(retained). 프로그램이 손으로 다시 그리는 것은 없다. 트리를 바꾸면 창이 바뀐 곳을 다시 그린다.

노드는 핸들 `gates_node_t` 로 부른다. 핸들은 칸 번호와 세대(generation)다. 노드가 없어지면 그 칸은 나중에 새 세대로 다른 노드를 담을 수 있으므로, 옛 핸들은 새 노드에 닿지 못한다. 모든 호출이 이를 확인하며, 낡은 핸들은 따르지 않고 거절한다(`PROVEN_ERR_INVALID_ARG`). 없애기는 하위 트리 전체를 한 번에 표시하고, 칸은 안전한 시점에 풀린다(`gates_tree_flush_destroys`, 창은 입력 차례마다 이것을 부른다). 그래서 처리기는 자기에게 이벤트를 보낸 노드를 없애도 된다.

```
/* manual example (host): handles, generations and copies.
 * expect: stale handle refused; label kept its copy */
#include <gates/gates.h>

#include <stdio.h>
#include <string.h>

int main(void) {
    gates_tree_t *tree = nullptr;
    if (!gates_is_ok(gates_tree_create(&(gates_tree_desc_t){0}, &tree))) return 1;
    gates_node_t root = gates_tree_root(tree), old, fresh;

    /* Text is copied: the caller's buffer may change or go away at once. */
    char buf[16] = "first";
    if (!gates_is_ok(gates_label_create(tree, root, (gates_str_t){ .ptr = (const gates_u8 *)buf, .size =
strlen(buf) }, &old))) return 1;
    strcpy(buf, "XXXXX");

    /* Destroying marks the node; its slot is freed at a safe point. */
    if (!gates_is_ok(gates_node_destroy(tree, old))) return 1;
    (void)gates_tree_flush_destroys(tree);

    /* A new node may reuse the slot, with a new generation: the old handle
     * never reaches it. */
    if (!gates_is_ok(gates_label_create(tree, root, GATES_STR("second"), &fresh))) return 1;
    bool stale_refused = !gates_node_is_valid(tree, old) &&
                         gates_widget_set_text(tree, old, GATES_STR("oops")) == PROVEN_ERR_INVALID_ARG;
    gates_str_t now = gates_widget_text(tree, fresh);
    bool kept = now.size == 6 && memcmp(now.ptr, "second", 6) == 0;
```

```
    printf("%s; %s\n", stale_refused ? "stale handle refused" : "STALE HANDLE WORKED",
           kept ? "label kept its copy" : "label changed");
    gates_tree_destroy(tree);
    return stale_refused && kept ? 0 : 1;
}
```

1.2 소유: 복사하는 것과 빌리는 것

프로그램이 gates 에 주는 문자열은 호출이 성공하면 모두 복사된다. 이름표, 글자, 명령 이름, 선택지 이름, 접근성 이름이 그렇다. 호출한 쪽의 버퍼는 곧바로 바꿔도 된다. gates 가 돌려주는 것은 빌린 것이다.

돌려받는 것	유효한 동안
gates_widget_text, gates_textbox_text	그 노드가 다음에 바뀔 때까지
이벤트의 ev->text	처리기가 돌아갈 때까지
뷰의 칸(gates_cell_t.text, 내 모델이 준 것)	모델을 다음에 부를 때까지
gates_access_info_t 의 문자열	그 트리에서 gates_access_info 를 다시 부르거나 무엇이든 바뀔 때까지

gates 에 등록한 콜백과 user 포인터(처리기, 모델, 명령 함수)는 등록된 동안 빌려 쓴다. 살려 두거나 먼저 등록을 푼다.

1.3 오류

실패할 수 있는 함수는 gates_err_t 를 돌려주고 [[nodiscard]] 가 붙어 있다. GATES_OK (gates_is_ok 로 본다)는 바꿈이 전부 일어났다는 뜻이고, 다른 값이면 아무것도 일어나지 않았다. 만들지 못한 노드는 반쯤 지은 것을 남기지 않는다. 오류 값은 proven 의 것이다. PROVEN_ERR_INVALID_ARG(틀렸거나 낡은 인자), PROVEN_ERR_INVALID_STATE(지금은 안 됨: 꺼진 컨트롤, 닫힌 대화상자), PROVEN_ERR_NOMEM, PROVEN_ERR_OUT_OF_BOUNDS(한도)가 있다. 메모리 부족은 충돌이 아니라 평범한 대답이며, 참조 응용들은 이를 처리한다. 크게 실패를 알릴 수 없는 입력(키, 클릭)은 그 실패를 gates_input_take_error 에 남긴다. 창의 on_input_error 콜백은 입력 한 차례가 끝날 때마다 그것을 듣고, 참조 응용들은 상태 줄에 보인다.

1.4 이벤트와 안전한 시점

프로그램은 사람이 한 일을 이벤트로 안다. 위젯마다 처리기가 많아야 하나 있다 (gates_widget_set_handler). 사람이 무언가를 바꾸면 gates 는 그것을 설명하는 이벤트를 대기열에 넣고, 창은 입력 메시지를 다 처리한 뒤의 안전한 시점에 대기열을 전달한다 (gates_tree_dispatch_events). 입력 처리 도중에는 아무것도 전달하지 않으므로, 처리기는 자기를 부른 노드를 포함해 무엇이든 바꾸거나 없애도 된다.

프로그램이 스스로 한 바꿈은 조용하다. 이미 알고 있기 때문이다. 나머지 프로그램이 들어야 할 때는 gates_widget_notify 가 출처가 GATES_ORIGIN_PROGRAM 인 이벤트를 넣는다.

```
/* manual example (host): events - what the person did, delivered at a safe point.
 * expect: user checked=1, program checked=0 */
#include <gates/gates.h>

#include <stdio.h>

static char log_text[128];
static int log_len;

static void on_value(gates_tree_t *tree, const gates_event_t *ev, void *user) {
    (void)tree;
    (void)user;
    if (ev->kind != GATES_EVENT_VALUE_CHANGED) return;
    log_len += snprintf(log_text + log_len, sizeof log_text - (size_t)log_len, "%s%s checked=%d",
                        log_len > 0 ? ", " : "", ev->origin == GATES_ORIGIN_USER ? "user" : "program",
```

```
                        ev->checked ? 1 : 0);
}

int main(void) {
    gates_tree_t *tree = nullptr;
    if (!gates_is_ok(gates_tree_create(&(gates_tree_desc_t){0}, &tree))) return 1;
    gates_node_t box;
    if (!gates_is_ok(gates_checkbox_create(tree, gates_tree_root(tree), GATES_STR("Send news"), false, nullptr,
nullptr, &box)) ||
        !gates_is_ok(gates_widget_set_handler(tree, box, on_value, nullptr))) {
        return 1;
    }
    /* A person ticks the box (here through the accessibility action, which
     * takes the same path as a click): an event is queued, not delivered. */
    if (!gates_is_ok(gates_access_toggle(tree, box))) return 1;
    /* The window delivers queued events after each input message; without a
     * window, the program does it. */
    (void)gates_tree_dispatch_events(tree, 0);

    /* Setters are silent: the program knows what it did. Announce it
     * explicitly when the rest of the program should hear it. */
    if (!gates_is_ok(gates_checkbox_set_checked(tree, box, false))) return 1;
    if (!gates_is_ok(gates_widget_notify(tree, box))) return 1;
    (void)gates_tree_dispatch_events(tree, 0);

    printf("%s\n", log_text);
    gates_tree_destroy(tree);
    return 0;
}
```

여기서 규칙 둘이 나온다. 무엇이 바뀌었는지 알려고 그리는 도중에 위젯 상태를 읽지 않는다. 이벤트를 따른다. 그리고 그리기 콜백 안에서 모델을 바꾸지 않는다. 그리기는 그리기만 한다.

1.5 단위

API 의 모든 좌표와 크기는 논리 단위, 1/96 인치다. 창은 가장자리에서 한 번 바꾼다. 자세한 것은 8장에 있고, 그때까지는 픽셀을 떠올릴 자리에서 "단위"로 읽으면 된다.

2 컨트롤과 배치

이 장의 차례

2.1 컨트롤 표 8
2.2 배치 10

헤더: `gates/widget.h`, `gates/layout.h`, `gates/geometry.h`.

2.1 컨트롤 표

컨트롤은 저마다 노드 하나다. 아래 표가 0.15.1 의 유한한 컨트롤 전부다. 사람이 그것으로 하는 일, 키, 프로그램이 받는 이벤트, 화면 읽기 프로그램이 듣는 것(9장)을 적었다.

컨트롤	만들기	사람은	키	이벤트	접근성으로는
패널	`gates_panel_create`	묶음을 본다	-	-	없음, 이름을 주면 그룹
이름표	`gates_label_create`	글을 읽는다	-	-	텍스트
버튼	`gates_button_create`	동작 하나를 청한다	Space(뗄 때), Enter	ACTIVATED	버튼 (Invoke)
체크박스	`gates_checkbox_create`	선택 사항을 켜고 끈다	Space	VALUE_CHANGED	체크박스 (Toggle)
텍스트 상자	`gates_textbox_create`	한 줄을 입력한다	편집 키, Ctrl+C/X/V/Z/Y, IME; 두 번 누르면 단어, 세 번 누르면 전부, Shift+클릭은 넓힌다	TEXT_CHANGED, PREEDIT_CHANGED, LIMIT_EXCEEDED	편집(Value, Text)
편집기	`gates_editor_create`	여러 줄을 쓴다	편집 키, Ctrl+화살표, PageUp/PageDown, Ctrl+C/X/V/Z/Y, IME; 청하면 Tab; 두 번 누르면 단어, 세 번 누르면 줄, Shift+클릭은 넓힌다	TEXT_CHANGED, SELECTION_CHANGED, PREEDIT_CHANGED	편집(Value, 줄 단위 Text)(6장)
라디오 그룹	`gates_radio_create`	몇 개 중 하나를 고른다	화살표, Home, End, Space	VALUE_CHANGED(result = 선택지 id)	(나의 결과)버튼의 그룹
선택 상자	`gates_choice_create`	목록에서 하나를 고른다	Space, Enter, Alt+Down 으로 열기; 화살표, Enter, Escape	VALUE_CHANGED	콤보 상자 (Selection, Expand)
진행 막대	`gates_progress_create`	일이 얼마나 되었는지 본다	-	-	진행 막대 (백분율)
구분선	`gates_separator_create`	나뉨을 본다	-	-	구분선

컨트롤	만들기	사람은	키	이벤트	접근성으로는
뷰	`gates_view_create`, `gates_log_create`	목록·표·트리·로그에서 고른다	화살표, PageUp/PageDown, Home, End, Enter; 트리는 Left/Right; 글자를 치면 그 줄로 간다; Ctrl+C 는 줄을 복사한다; Ctrl+Left/Right 는 F2 와 Space 가 쓰는 열을 고른다; multi_select 이면 Shift/Ctrl 로 여럿, Ctrl+A 로 전부	SELECTION_CHANGED, ACTIVATED, SORT_REQUESTED, EXPAND_REQUESTED, CELL_EDITED	목록, 표, 트리(5장)
폼	`gates_form_create`	이름표 달린 필드를 채운다	-	편집기들의 이벤트	필드의 그룹(3장)
속성 격자	`gates_propgrid_create`	레코드의 형식 있는 필드를 고친다	Tab, 편집기들의 키	VALUE_CHANGED(id = 속성 id)	(이름, 값) 편집기의 그룹(5장)
대화상자, 메뉴	`gates_dialog_open`, `gates_menu_open`	한 번 답한다, 명령을 고른다	Enter, Escape, 화살표	DIALOG_CLOSED, MENU_CLOSED	창, 메뉴(4장)
메뉴 막대	`gates_menubar_create`	메뉴에서 명령을 고른다	F10 또는 Alt, 화살표, 글자, Escape	MENU_CLOSED, 명령	메뉴 막대(11장)
도구 막대	`gates_toolbar_create`	한 번 클릭으로 명령을 실행한다	화살표, Space, Enter	명령	도구 막대(11장)
상태 줄	`gates_statusbar_create`	프로그램의 상태를 읽는다	-	-	상태 줄(11장)
스핀 상자	`gates_spin_create`	숫자를 입력하거나 한 칸씩 바꾼다	Up/Down, PgUp/PgDn, Enter; 화살표를 누르고 있으면 되풀이; 포커스가 있을 때 휠	VALUE_CHANGED(value)	(Value) (RangeValue) (12장)
슬라이더	`gates_slider_create`	트랙을 따라 숫자를 끈다	화살표, PgUp/PgDn, Home, End; 포커스가 있을 때 휠	VALUE_CHANGED(value)	(Value) (RangeValue) (12장)
그룹 상자	`gates_group_create`	컨트롤 묶음을 보고 접는다	제목에서 Space, Enter	VALUE_CHANGED(1 = 펼침)	(Value) (Checked) (ExpandCollapse) (12장)
이미지	`gates_image_create`	그림을 본다	-	-	이름이 있으면 이미지(13장)
탭	`gates_tabs_create`	페이지 사이를 오간다	화살표, Ctrl+Tab, Ctrl+PgUp/PgDn	VALUE_CHANGED(값 = 번호)	(Value) (Selection) (11장)

Tab 과 Shift+Tab 은 켜져 있고 보이는 컨트롤을 트리 차례대로 오간다. `gates_widget_set_focusable` 로 차례에서 뺄 수 있다. 끈 컨트롤(`gates_widget_set_disabled`)은 흐리게 그려지고, 입력을 무시하고, Tab 이 건너뛴다. 숨긴 노드(`gates_node_set_hidden`)와 그 아래 전부는 자리를 차지하지 않고, 그려지지 않고, 닿을 수 없다.

라디오 그룹과 선택 상자는 선택지를 자리가 아닌 변하지 않는 id(`gates_option_t`)로 부른다. id 0 은 "고른 것 없음"이다.

만들기 함수의 옛 `on_click` / `on_toggle` 인자는 입력 전달 안에서 돌며 호환을 위해 남아 있다. 새 코드는 null 을 주고 처리기를 쓴다.

2.2 배치

배치(layout)는 내재적이다. 컨트롤은 저마다 원하는 크기를 알고, 담는 쪽이 자식을 놓는다. 프로그램은 담는 쪽의 종류를 고른다(`gates_layout_set`).

종류	자식을 놓는 법
`GATES_LAYOUT_KIND_COLUMN`	위에서 아래로, 저마다 세로줄만큼 넓게
`GATES_LAYOUT_KIND_ROW`	왼쪽에서 오른쪽으로, 저마다 가로줄만큼 높게
`GATES_LAYOUT_KIND_STACK`	겹쳐서, 활성인 하나만 보인다(쪽)
`GATES_LAYOUT_KIND_SPLIT`	두 칸과 사람이 끄는 손잡이
`GATES_LAYOUT_KIND_SCROLL`	휠과 스크롤 막대가 있는 창구 안의 세로줄(트랙을 누르면 한 쪽씩)
`GATES_LAYOUT_KIND_FORM`	줄마다 이름표 옆에 편집기(3장)
`GATES_LAYOUT_KIND_ABSOLUTE`	프로그램이 준 사각형에

안쪽 여백(padding)과 간격(gap)은 담는 쪽마다 정한다. `gates_layout_set_child_grow` 는 남는 공간의 몫을 자식에게 주고, `gates_layout_set_child_align` 은 늘어나지 않는 자식의 자리를 정한다. 보통 흐름에서 자식들은 겹치지 않는다. 창은 그리기 전에 논리 단위로 트리를 배치한다. 창이 없으면 `gates_layout_run` 이 한다.

넘겨주는 숫자에는 어떤 창도 닿지 않을 한계가 있다(0.15.0). 안쪽 여백이나 간격이 `GATES_EXTENT_MAX`(65,536)를 넘으면 음수처럼 거절되고, 뷰포트·절대 사각형·스크롤 위치·포인터 위치는 +-`GATES_COORD_MAX`(16,777,216)로 제한된다. 배치 계산이 32비트 안에 머물게 하려는 것이다.

마우스 포인터는 누르면 무슨 일이 일어날지 보여 준다(0.13.0). 분할 핸들과 표의 열 경계 위에서는 크기 조절 화살표(좌우, 위아래로 쌓인 창이면 상하)로, 텍스트 상자와 편집기의 글자 위에서는 I자 모양으로 바뀌고, 끄는 동안 내내 그 모양을 유지한다. 창은 이것을 스스로 한다. 직접 만든 플랫폼에서는 `gates_cursor_at(tree, point)` 가 같은 물음에 답한다.

```c
/* manual example (host): layout - a column with a growing row.
 * expect: the list takes the rest; buttons in a row */
#include <gates/gates.h>

#include <stdio.h>

#define TRY(x) do { if (!gates_is_ok(x)) return 1; } while (0)

int main(void) {
    gates_tree_t *t = nullptr;
    TRY(gates_tree_create(&(gates_tree_desc_t){0}, &t));
    gates_node_t root = gates_tree_root(t), title, list, bar, ok, cancel;
    TRY(gates_layout_set(t, root, GATES_LAYOUT_KIND_COLUMN));
    TRY(gates_layout_set_padding(t, root, 8));   /* logical units: 1/96 inch */
    TRY(gates_layout_set_gap(t, root, 6));
    TRY(gates_label_create(t, root, GATES_STR("Files"), &title));
    TRY(gates_panel_create(t, root, &list));
    TRY(gates_layout_set_child_grow(t, list, 1)); /* takes the space left over */
    TRY(gates_panel_create(t, root, &bar));
    TRY(gates_layout_set(t, bar, GATES_LAYOUT_KIND_ROW));
    TRY(gates_layout_set_gap(t, bar, 6));
    TRY(gates_button_create(t, bar, GATES_STR("OK"), nullptr, nullptr, &ok));
    TRY(gates_button_create(t, bar, GATES_STR("Cancel"), nullptr, nullptr, &cancel));
```

```
    TRY(gates_layout_run(t, (gates_size_t){ 240, 320 }, gates_text_backend_builtin()));

    gates_rect_t l = gates_node_layout_rect(t, list), a = gates_node_layout_rect(t, ok),
                 b = gates_node_layout_rect(t, cancel), r = gates_node_layout_rect(t, bar);
    /* The list ends one gap above the button row, which ends at the padding. */
    bool rest = l.y + l.h + 6 == r.y && r.y + r.h == 320 - 8;
    printf("%s; buttons %s\n", rest ? "the list takes the rest" : "the list does not grow",
           a.y == b.y && b.x > a.x ? "in a row" : "not in a row");
    gates_tree_destroy(t);
    return 0;
}
```

스크롤 담기는 키보드 포커스를 따라간다. 창구 아래의 컨트롤로 Tab 하면 보이는 곳으로 스크롤된다. 따로 청하지 않으면 위아래로만 스크롤한다(0.10.0). `gates_layout_set_scroll_sideways(tree, area, true)` 를 주면 자식들을 그 가운데 가장 넓은 것만큼 넓게 놓고, 그것이 창구보다 넓으면 아래쪽에 가로 막대가 나타난다. 휠의 옆 움직임(Windows 에서는 Shift+휠)이 이를 움직이고, Tab 은 포커스 받은 컨트롤을 가로로도 보이게 하며, 보조 기술도 스크롤할 수 있다. 위치는 `gates_layout_set_scroll_x`, `gates_layout_scroll_x` 로 쓰고 읽는다. 폼에는 켜지 않는다. 자식이 창구 너비와 같은 것이 폼에 맞다. 활성이 아닌 겹침(stack) 쪽은 그려지지도 닿지도 않는다.

3 폼

이 장의 차례

3.1 폼은 데이터를 갖지 않는다 12
3.2 초안 검사 13

헤더: `gates/form.h`.

3.1 폼은 데이터를 갖지 않는다

폼은 이름표 달린 필드를 배치한다. 편집기 옆에 이름표, 그 아래 도움말 줄과 오류 줄을 둔다. 그 밖의 일은 하지 않는다. 값을 갖지도, 검사하지도, 저장하지도 않는다. 응용이 초안(draft)을 갖고, 편집기를 읽거나 그 이벤트를 따르고, 무엇이 맞는지 정하고, 필드에 메시지를 달고, 제출이 성공한 뒤에 저장한다. 이 나눔이 요점이다. 맞는 데이터의 규칙은 프로그램의 것이고, 필드와 오류를 보여 주는 일은 gates 의 것이어서, 둘이 누가 진실을 쥐었는지 다투지 않는다.

필드마다 프로그램이 고른 id(`field_id`, 0 이 아니고 폼 안에서 유일)가 있다. 줄을 숨기거나 순서를 바꿔도 id 는 그대로다. `gates_form_editor` 가 필드의 편집기를 준다.

더하기	편집기	값
`gates_form_add_text`	텍스트 상자	`gates_textbox_text`
`gates_form_add_checkbox`	체크박스	`gates_checkbox_checked`
`gates_form_add_choice`	선택 상자	VALUE_CHANGED 의 선택지 id
`gates_form_add_radio`	라디오 그룹	VALUE_CHANGED 의 선택지 id

`required` 는 이름표에 표시를 붙이고 필드가 필수임을 보조 기술에 알린다. `help` 는 편집기 아래의 한 줄이다. `gates_form_set_error` 는 필드 아래에 오류 색으로 메시지를 보이고 텍스트 상자를 틀린 상태로 표시한다. 빈 메시지는 지운다. 좁은 폼은 이름표를 편집기 옆이 아니라 위에 두고, 그렇게 쌓인 줄만큼 높아지므로 그 아래 것은 가려지지 않고 밑으로 내려간다.

```
/* manual example (host): a form, a draft, validation on submit.
 * expect: first submit: 1 error; second submit: saved Kim */
#include <gates/gates.h>

#include <stdio.h>

enum { FIELD_NAME = 1, FIELD_NEWS = 2 };

/* The application's draft: the form holds no data of its own. */
typedef struct draft_t {
    char name[32];
    bool news;
} draft_t;

/* Reads the editors, checks, and puts messages on the fields. */
static int submit(gates_tree_t *t, gates_node_t form, draft_t *d) {
    gates_str_t name = gates_textbox_text(t, gates_form_editor(t, form, FIELD_NAME));
    int errors = 0;
    if (name.size == 0) {
        (void)gates_form_set_error(t, form, FIELD_NAME, GATES_STR("enter a name"));
        errors++;
```

```
    } else {
        (void)gates_form_set_error(t, form, FIELD_NAME, GATES_STR("")); /* clears it */
        snprintf(d->name, sizeof d->name, "%.*s", (int)name.size, (const char *)name.ptr);
    }
    d->news = gates_checkbox_checked(t, gates_form_editor(t, form, FIELD_NEWS));
    return errors;
}

int main(void) {
    gates_tree_t *t = nullptr;
    if (!gates_is_ok(gates_tree_create(&(gates_tree_desc_t){0}, &t))) return 1;
    gates_node_t form, name_box;
    gates_field_desc_t name = { .label = GATES_STR("Name"), .required = true, .max_bytes = 30,
                                .help = GATES_STR("as it should appear on the badge") };
    gates_field_desc_t news = { .label = GATES_STR("News"), .text = GATES_STR("send me news") };
    if (!gates_is_ok(gates_form_create(t, gates_tree_root(t), &form)) ||
        !gates_is_ok(gates_form_add_text(t, form, FIELD_NAME, &name, &name_box)) ||
        !gates_is_ok(gates_form_add_checkbox(t, form, FIELD_NEWS, &news, nullptr))) {
        return 1;
    }
    draft_t d = {0};
    int first = submit(t, form, &d);
    /* The error is part of the field now: screen readers read it with the name. */
    gates_access_info_t info;
    if (!gates_is_ok(gates_access_info(t, name_box, 0, &info)) || info.description.size == 0) return 1;

    if (!gates_is_ok(gates_textbox_set_text(t, name_box, GATES_STR("Kim")))) return 1;
    int second = submit(t, form, &d);
    printf("first submit: %d error%s; second submit: %s %s\n", first, first == 1 ? "" : "s",
           second == 0 ? "saved" : "refused", d.name);
    gates_tree_destroy(t);
    return 0;
}
```

3.2 초안 검사

검사는 키마다가 아니라 제출할 때 한다. 전자우편 주소를 반쯤 쓴 사람은 아직 틀리지 않았다. 제출이 실패하면 오류를 모두 제 필드에 달고, 첫 오류를 고칠 수 있는 곳에 포커스를 두고, 무슨 일이 있었는지 상태 줄에 말한다(9장이 그 줄이 스스로 알리게 하는 법을 보인다). 필드의 오류는 사람이 그 필드를 바꿀 때나 다음 제출 때 지운다.

참조 응용 `app_settings` 가 이 모두를 한다. 필수 필드, 도움말, 오류, 바꾼 것 버리기, 맞을 때만 초안 저장하기가 들어 있다.

4 명령, 포커스, 대화상자, 메뉴

이 장의 차례

4.1 동작 하나, 청하는 길 여럿 14
4.2 포커스 15
4.3 대화상자 15
4.4 문맥 메뉴 15
4.5 실행 취소와 다시 실행 15

헤더: `gates/command.h`, `gates/overlay.h`.

4.1 동작 하나, 청하는 길 여럿

"저장"은 사람이 버튼을 누르든, Ctrl+S 를 누르든, 문맥 메뉴에서 고르든 같은 동작 하나다. 명령 (command) 이 이것을 한 번만 갖는다. id, 이름, 단축키(있으면), 켜짐과 체크 상태, 실행할 함수다. 명령은 범위(scope) 노드에 속하고, 단축키는 포커스가 그 범위 안에 있을 때 동작한다. 명령에 묶인 버튼(`gates_button_set_command`)은 명령의 이름과 켜짐 상태를 가져오고, 명령이 바뀌면 따라간다. 명령을 끄면 그것을 부르는 모든 길이 한꺼번에 꺼진다.

부르기는 대기열에 들어가고 다음 안전한 시점에, 명령을 다시 확인한 뒤에 돈다. 그 사이에 꺼진 명령은 돌지 않는다.

```c
/* manual example (host): one command for a button and a shortcut.
 * expect: saved 1 time; button disabled with the command */
#include <gates/gates.h>

#include <stdio.h>

enum { CMD_SAVE = 1 };

static void on_save(gates_tree_t *tree, gates_command_id_t id, void *user) {
    (void)tree;
    (void)id;
    (*(int *)user)++;
}

int main(void) {
    gates_tree_t *t = nullptr;
    if (!gates_is_ok(gates_tree_create(&(gates_tree_desc_t){0}, &t))) return 1;
    gates_node_t root = gates_tree_root(t), save;
    int saves = 0;
    gates_command_desc_t cmd = { .id = CMD_SAVE, .label = GATES_STR("Save"), .enabled = true,
                                 .shortcut = { .letter = 'S', .ctrl = true },
                                 .invoke = on_save, .user = &saves };
    if (!gates_is_ok(gates_command_register(t, root, &cmd)) ||
        !gates_is_ok(gates_button_create(t, root, GATES_STR(""), nullptr, nullptr, &save)) ||
        !gates_is_ok(gates_button_set_command(t, save, root, CMD_SAVE))) { /* label, state from the command */
        return 1;
    }
    /* Ctrl+S: the command is queued and runs at the next safe point. */
    gates_key_event_t ctrl_s = { .letter = 'S', .ctrl = true, .down = true };
    (void)gates_input_key(t, &ctrl_s);
    (void)gates_tree_dispatch_events(t, 0);
```

```
    /* Disabling the command disables every button bound to it, and its shortcut. */
    if (!gates_is_ok(gates_command_set_enabled(t, root, CMD_SAVE, false))) return 1;
    (void)gates_input_key(t, &ctrl_s);
    (void)gates_tree_dispatch_events(t, 0);

    gates_access_info_t info;
    if (!gates_is_ok(gates_access_info(t, save, 0, &info))) return 1;
    printf("saved %d time%s; button %s with the command\n", saves, saves == 1 ? "" : "s",
           (info.states & GATES_ACCESS_DISABLED) != 0 ? "disabled" : "enabled");
    gates_tree_destroy(t);
    return 0;
}
```

명령에는 역할이 있을 수 있다. 범위의 DEFAULT 명령은 그 범위의 텍스트 상자나 체크박스에서 Enter 를 누르면 돌고, CANCEL 명령은 범위 어디서든 Escape 로 돈다(대화상자라면 확인과 취소).

4.2 포커스

키보드 포커스는 컨트롤 하나에 있다(`gates_tree_focus`, `gates_tree_set_focus`). Tab 과 Shift+Tab 은 트리 차례로 옮기며 지금의 포커스 범위 안에서 돈다. 창의 범위는 트리 전체이고, 열린 대화상자의 범위는 그 대화상자다. 포커스를 가진 컨트롤이 숨겨지거나 없어지면 포커스를 놓는다(그 안의 누름과 끌기도 놓는다). 포커스를 다음에 어디 둘지는 프로그램이 `gates_tree_set_focus` 로 정한다. 대화상자가 닫히면 포커스는 그 전에 가졌던 컨트롤로 돌아간다.

4.3 대화상자

`gates_dialog_open` 은 창 안에 모달 대화상자를 보인다. 창의 나머지는 흐려지고 입력을 무시하며, Tab 은 안에만 머물고, 창 자신의 단축키는 기다린다. 열기는 곧바로 돌아온다. 중첩된 이벤트 루프는 없다. 프로그램은 대화상자의 내용 패널을 채우고, 대화상자를 범위로 삼아 명령을 등록하고, 답을 DIALOG_CLOSED 이벤트 한 번(`ev->result`: ACCEPTED 또는 CANCELED)으로 안다. 포커스는 원래 자리로 돌아간다. 대화상자와 메뉴는 한 번에 여덟 개까지 열린다.

4.4 문맥 메뉴

`gates_menu_open` 은 범위의 명령들을 한 점에 나열한다. 이름, 체크 표시, 단축키, 켜짐 상태는 명령에서 온다. 항목을 고르면 단축키와 같은, 대기열을 거치고 다시 확인하는 길로 명령을 부른다. 화살표가 옮기고, Enter 가 고르고, Escape 나 바깥 누름은 닫기만 한다(바깥을 누른 것이 그 아래 것을 함께 누르는 일은 없다).

`gates_command_set_submenu(tree, scope, id, ids, count)` 는 항목에 같은 범위의 다른 명령들로 된 하위 메뉴를 준다. 그 항목에는 화살표가 붙고, Right, Enter, 클릭, 또는 포인터를 잠시 올려 두면 옆에 하위 메뉴가 열리며, Left 나 Escape 가 다시 닫는다. 하위 메뉴에서 고르면 그 명령을 부르고 줄지어 열린 메뉴를 모두 닫으며, 처음 연 메뉴가 그 명령 id 로 MENU_CLOSED 를 알린다. 메뉴 막대의 메뉴에서도 같다.

4.5 실행 취소와 다시 실행

글 상자는 자기 입력을 스스로 되돌린다. 프로그램 자신의 데이터(이름을 바꾼 항목, 옮긴 줄, 바꾼 설정)에는 `gates_undo_create`(gates/undo.h)가 스택을 준다. 바꾼 뒤 이름표와 두 함수(되돌리기, 다시 하기)를 담은 항목을 넣는다. 같은 합치기 키로 이어진 항목은 하나가 되고(한 칸에 글 치기), 스택은 정해진 개수만 갖고, `gates_undo_mark_clean` / `gates_undo_is_clean` 은 데이터가 저장한 그대로인지 알려 준다. `gates_undo_bind` 는 명령 둘을 맞춰 둔다. 되돌리거나 다시 할 것이 있을 때만 켜지고, 이름은 "Undo Rename" / "Redo Rename" 처럼 된다(낱말은 프로그램의 언어로 줄 수 있다).

```
/* manual example (host): Undo and Redo commands over the program's own data.
 * expect: menu shows "Undo Rename"; after Undo: name Draft, menu shows "Redo Rename"; clean again: yes */
#include <gates/gates.h>
```

```
#include <stdio.h>
#include <string.h>

enum { CMD_UNDO = 1, CMD_REDO = 2 };

static char name[32] = "Draft";

/* One entry remembers the name before and after the change. */
typedef struct rename_t {
    char before[32], after[32];
} rename_t;

static gates_err_t undo_rename(void *data) {
    memcpy(name, ((rename_t *)data)->before, sizeof name);
    return GATES_OK;
}
static gates_err_t redo_rename(void *data) {
    memcpy(name, ((rename_t *)data)->after, sizeof name);
    return GATES_OK;
}

static void on_command(gates_tree_t *tree, gates_command_id_t id, void *user) {
    (void)tree;
    gates_undo_t *undo = user;
    (void)(id == CMD_UNDO ? gates_undo_undo(undo) : gates_undo_redo(undo));
}

static void print_label(gates_tree_t *t, gates_command_id_t id) {
    gates_str_t l = gates_command_label(t, gates_tree_root(t), id);
    printf("\"%.*s\"", (int)l.size, (const char *)l.ptr);
}

int main(void) {
    gates_tree_t *t = nullptr;
    gates_undo_t *undo = nullptr;
    if (!gates_is_ok(gates_tree_create(&(gates_tree_desc_t){0}, &t)) ||
        !gates_is_ok(gates_undo_create((gates_allocator_t){0}, 0, &undo))) {
        return 1;
    }
    gates_node_t root = gates_tree_root(t);
    gates_command_desc_t cmds[] = {
        { .id = CMD_UNDO, .label = GATES_STR_INIT("Undo"), .shortcut = { .key = GATES_KEY_Z, .ctrl = true },
          .enabled = true, .invoke = on_command, .user = undo },
        { .id = CMD_REDO, .label = GATES_STR_INIT("Redo"), .shortcut = { .key = GATES_KEY_Y, .ctrl = true },
          .enabled = true, .invoke = on_command, .user = undo },
    };
    if (!gates_is_ok(gates_command_register(t, root, &cmds[0])) || !gates_is_ok(gates_command_register(t, root,
&cmds[1])) ||
        !gates_is_ok(gates_undo_bind(undo, t, root, CMD_UNDO, CMD_REDO, GATES_STR("Undo"), GATES_STR("Redo")))) {
        return 1;
    }
    gates_undo_mark_clean(undo); /* the name as saved */

    /* The program renames, then records how to take it back. */
    static rename_t change = { "Draft", "Final" };
    memcpy(name, change.after, sizeof name);
    gates_undo_entry_t entry = { .label = GATES_STR_INIT("Rename"), .undo = undo_rename, .undo_data = &change,
                                 .redo = redo_rename, .redo_data = &change };
    if (!gates_is_ok(gates_undo_push(undo, &entry))) return 1;
    printf("menu shows ");
    print_label(t, CMD_UNDO);
```

```
    /* A person presses Ctrl+Z. */
    gates_key_event_t ctrl_z = { .key = GATES_KEY_Z, .ctrl = true, .down = true };
    (void)gates_input_key(t, &ctrl_z);
    (void)gates_tree_dispatch_events(t, 0);
    printf("; after Undo: name %s, menu shows ", name);
    print_label(t, CMD_REDO);
    printf("; clean again: %s\n", gates_undo_is_clean(undo) ? "yes" : "no");

    gates_undo_destroy(undo);
    gates_tree_destroy(t);
    return 0;
}
```

5 내 데이터 위의 뷰

이 장의 차례

5.1 줄이 몇 개든 노드 하나 18
5.2 목록, 표, 트리, 로그 19
5.3 사람이 바꿀 수 있는 칸 20
5.4 여러 줄을 한꺼번에: 다중 선택 22
5.5 한 레코드의 필드: 속성 격자 24
5.6 모델의 규칙 25

헤더: `gates/view.h`.

5.1 줄이 몇 개든 노드 하나

백만 줄짜리 목록도 노드 하나다. 줄은 응용의 것이다. 응용은 모델로 줄을 설명한다. 몇 줄인지, 한 자리의 id 가 무엇인지, id 의 자리가 어디인지, 칸의 글이 무엇인지를 말한다. gates 는 보이는 줄만 묻는다(한 번에 `GATES_VIEW_MAX_ROWS` 줄까지). gates 로 복사되는 것은 없고, 백만 줄을 스크롤하는 비용은 한 화면을 스크롤하는 비용과 같다.

줄에는 id(`gates_item_id_t`, 0 이 아니고 변하지 않음)가 있다. 고른 것(selection)은 자리가 아니라 id 이므로, 위에 줄이 끼어들거나 다른 곳이 지워지거나 정렬되어도 같은 줄에 머문다. 응용은 줄을 바꾼 뒤 `gates_view_model_changed` 를 부른다. 고른 줄이 없어졌으면 고른 것이 가장 가까운 남은 줄로 옮겨 가고, 출처가 PROGRAM 인 SELECTION_CHANGED 가 이를 알린다.

```
/* manual example (host): a list over the application's own rows.
 * expect: selected 3 (Cherry); after removing it: 4 (Damson), origin program */
#include <gates/gates.h>

#include <stdio.h>
#include <string.h>

typedef struct row_t {
    gates_item_id_t id;          /* stable: survives inserts, removals, sorting */
    const char *name;
} row_t;

typedef struct fruits_t {
    row_t rows[8];
    gates_u64 n;
} fruits_t;

/* The model: gates asks only for the rows it shows, by position and by id. */
static gates_u64 f_count(void *u) { return ((fruits_t *)u)->n; }
static gates_item_id_t f_id_at(void *u, gates_u64 row) { return ((fruits_t *)u)->rows[row].id; }
static bool f_index_of(void *u, gates_item_id_t id, gates_u64 *row) {
    fruits_t *f = u;
    for (gates_u64 i = 0; i < f->n; i++) {
        if (f->rows[i].id == id) { *row = i; return true; }
    }
    return false;
}
static gates_err_t f_cell(void *u, gates_item_id_t id, gates_column_id_t col, gates_cell_t *out) {
    (void)col;
```

```
    gates_u64 row;
    if (!f_index_of(u, id, &row)) return PROVEN_ERR_INVALID_ARG;
    const char *s = ((fruits_t *)u)->rows[row].name;
    out->text = (gates_str_t){ .ptr = (const gates_u8 *)s, .size = strlen(s) };
    return GATES_OK;
}

static gates_item_id_t last_id;
static gates_event_origin_t last_origin;

static void on_view(gates_tree_t *tree, const gates_event_t *ev, void *user) {
    (void)tree;
    (void)user;
    if (ev->kind == GATES_EVENT_SELECTION_CHANGED) {
        last_id = ev->item;
        last_origin = ev->origin;
    }
}

static const char *name_of(fruits_t *f, gates_item_id_t id) {
    gates_u64 row;
    return f_index_of(f, id, &row) ? f->rows[row].name : "none";
}

int main(void) {
    fruits_t f = { .rows = { { 1, "Apple" }, { 2, "Banana" }, { 3, "Cherry" }, { 4, "Damson" }, { 5,
"Elder" } }, .n = 5 };
    gates_tree_t *t = nullptr;
    if (!gates_is_ok(gates_tree_create(&(gates_tree_desc_t){0}, &t))) return 1;
    gates_node_t list;
    gates_rows_model_t model = { .user = &f, .count = f_count, .id_at = f_id_at, .index_of = f_index_of, .cell =
f_cell };
    if (!gates_is_ok(gates_view_create(t, gates_tree_root(t), &(gates_view_desc_t){0}, &list)) ||
        !gates_is_ok(gates_view_set_model(t, list, &model)) ||
        !gates_is_ok(gates_widget_set_handler(t, list, on_view, nullptr)) ||
        !gates_is_ok(gates_layout_run(t, (gates_size_t){ 200, 160 }, gates_text_backend_builtin()))) {
        return 1;
    }
    /* A person picks the third row. */
    if (!gates_is_ok(gates_access_select(t, list, 3))) return 1;
    (void)gates_tree_dispatch_events(t, 0);
    printf("selected %llu (%s); ", (unsigned long long)last_id, name_of(&f, last_id));

    /* The application removes that row, then tells the view its rows changed:
     * the selection moves to the nearest remaining row and says so. */
    memmove(&f.rows[2], &f.rows[3], 2 * sizeof f.rows[0]);
    f.n = 4;
    if (!gates_is_ok(gates_view_model_changed(t, list))) return 1;
    (void)gates_tree_dispatch_events(t, 0);
    printf("after removing it: %llu (%s), origin %s\n", (unsigned long long)last_id, name_of(&f, last_id),
           last_origin == GATES_ORIGIN_PROGRAM ? "program" : "user");
    gates_tree_destroy(t);
    return 0;
}
```

5.2 목록, 표, 트리, 로그

- 목록에는 열이 없다. 묻는 칸은 열 0 이다.
- 표에는 열(gates_column_desc_t: id, 이름, 너비, 최소 너비)이 있고 머리줄을 보일 수 있다. 머리 칸을 누르면 열 id 와 함께 SORT_REQUESTED 가 온다. 응용이 줄을 정렬하고 gates_view_model_changed 를 부른다. 머리 칸의 가장자리를 끌면 열 너비가 바뀐다. gates_view_set_column_hidden 과 gates_view_move_column

으로 열을 고르고 순서를 정한다. 설명에 column_menu 를 주면 머리줄을 오른쪽 단추로 누르거나(표에서는 Shift+F10) 열마다 체크 항목이 있는 메뉴가 열리고, 마지막으로 보이는 열은 숨길 수 없다. gates/state.h 는 너비, 순서, 숨긴 열을 저장한다.

- 트리는 모델이 row_info 까지 답하는 목록이다. 깊이, 열 수 있는지, 열려 있는지, 불러오는 중인지 오류인지를 답한다. 열기와 닫기는 요청(EXPAND_REQUESTED)이다. 응용이 펼친 줄들을 바꾸고 gates_view_model_changed 를 부른다. gates 는 보이지 않는 줄을 훑지 않는다.
- 로그(gates_log_create)는 줄을 스스로 갖는다. 글을 덧붙이고, 줄 수와 바이트 한도를 지키고, 가장 오래된 줄부터 버리고, 사람이 스크롤해 떠나지 않은 동안 끝을 따라간다. 스크롤해 떠나거나 돌아오면 GATES_EVENT_FOLLOW_CHANGED 를 보내므로(따라가는 동안 ev->result 는 1) 상태 줄은 포인터를 지켜보지 않고도 그것을 말할 수 있다.

모든 뷰는 입력에도 답한다. 현재 열(테두리가 그려진 열. Ctrl+Left/Right 나 클릭으로 옮긴다)의 칸이, 현재 열이 없으면 첫 보이는 칸이 친 글자로 시작하는 다음 줄이 선택된다(0.10.0. 1초 쉬면 새로 찾고, 같은 글자를 다시 치면 그런 줄을 차례로 돈다). Ctrl+C 는 선택한 줄의 보이는 칸들을 탭으로 나눠 클립보드에 넣는다.

5.3 사람이 바꿀 수 있는 칸

열은 글 말고도 보여 줄 수 있다(gates_column_desc_t.kind). GATES_CELL_CHECK 는 cell.checked 로 체크 상자를, GATES_CELL_PROGRESS 는 cell.permille(0~1000)로 막대를, GATES_CELL_ICON_TEXT 는 글 앞에 16 x 16 아이콘(cell.icon, 13장)을 그린다. paint 함수를 준 열은 칸마다 그 칸 안에서 스스로 그린다.

editable 로 표시한 열은 사람이 선택한 줄의 칸을 바꿀 수 있게 하고, 모델은 set_cell 함수로 받아들일지 정한다. F2 나 두 번 누르기는 칸 위에 모델의 글을 선택한 채로 글 상자를 연다. Enter 는 확정하고, Escape 는 취소하고, 포커스가 떠나면 확정한다. 체크 열은 Space 나 상자를 누르면 바뀐다. F2 와 Space 가 쓰는 열은 표의 현재 열(마지막으로 누른 칸이나 Ctrl+Left·Ctrl+Right 로 옮긴 열, 선택한 줄에 테두리로 보인다)이 맞는 종류의 고칠 수 있는 열이면 그 열이고, 아니면 그런 첫 열이다. set_cell 이 오류를 돌려주면 편집기는 열린 채 잘못됨으로 표시된다(포커스가 이미 떠났다면 그 편집은 버린다). 바뀐 뒤 뷰는 모델을 다시 읽고 줄(ev->item)과 열(ev->result)을 담아 CELL_EDITED 를 보낸다. 프로그램은 gates_view_edit 와 gates_view_end_edit 로 같은 일을 할 수 있다.

화면 낭독기에는 표 줄의 칸 하나하나가 따로 된 요소로 보인다. 칸의 글자, 그 열의 이름, 체크 칸의 상태, 진행 칸의 백분율이다. 열이 고칠 수 있으면 체크 칸을 바꾸거나 글자 칸의 값을 정할 수 있고, 이것은 set_cell 을 거쳐 CELL_EDITED 로 알려지며 사람의 편집과 똑같이 거절될 수 있다(gates_access_cell_info, _toggle, _set_value). 보기보다 넓은 표는 화면 낭독기에서도 옆으로 스크롤된다.

```
/* manual example (host): a shopping list edited in place - a text column and a check column.
 * expect: item 2 is now Bread (column 1), done: yes (column 2); refused empty name: editor still open */
#include <gates/gates.h>

#include <stdio.h>
#include <string.h>

enum { COL_NAME = 1, COL_DONE = 2 };

typedef struct item_t {
    gates_item_id_t id;
    char name[32];
    bool done;
} item_t;

typedef struct list_t {
    item_t items[4];
    gates_u64 n;
} list_t;

static gates_u64 l_count(void *u) { return ((list_t *)u)->n; }
static gates_item_id_t l_id_at(void *u, gates_u64 row) { return ((list_t *)u)->items[row].id; }
```

```
static bool l_index_of(void *u, gates_item_id_t id, gates_u64 *row) {
    list_t *l = u;
    for (gates_u64 i = 0; i < l->n; i++) {
        if (l->items[i].id == id) { *row = i; return true; }
    }
    return false;
}
static gates_err_t l_cell(void *u, gates_item_id_t id, gates_column_id_t col, gates_cell_t *out) {
    gates_u64 row;
    if (!l_index_of(u, id, &row)) return PROVEN_ERR_INVALID_ARG;
    item_t *it = &((list_t *)u)->items[row];
    if (col == COL_DONE) {
        out->checked = it->done;
    } else {
        out->text = (gates_str_t){ .ptr = (const gates_u8 *)it->name, .size = strlen(it->name) };
    }
    return GATES_OK;
}
/* A person changed a cell: the model decides. An empty name is refused. */
static gates_err_t l_set_cell(void *u, gates_item_id_t id, gates_column_id_t col, const gates_cell_t *value) {
    gates_u64 row;
    if (!l_index_of(u, id, &row)) return PROVEN_ERR_INVALID_ARG;
    item_t *it = &((list_t *)u)->items[row];
    if (col == COL_DONE) {
        it->done = value->checked;
        return GATES_OK;
    }
    if (value->text.size == 0 || value->text.size >= sizeof it->name) return PROVEN_ERR_INVALID_ARG;
    memcpy(it->name, value->text.ptr, value->text.size);
    it->name[value->text.size] = 0;
    return GATES_OK;
}

static list_t list = { .items = { { 1, "Milk", false }, { 2, "Bred", false }, { 3, "Eggs", true } }, .n = 3 };

static void on_view(gates_tree_t *tree, const gates_event_t *ev, void *user) {
    (void)tree;
    (void)user;
    if (ev->kind != GATES_EVENT_CELL_EDITED) return;
    gates_u64 row;
    if (!l_index_of(&list, ev->item, &row)) return;
    if (ev->result == COL_NAME) {
        printf("item %llu is now %s (column %u), ", (unsigned long long)ev->item, list.items[row].name, ev-
>result);
    } else {
        printf("done: %s (column %u); ", list.items[row].done ? "yes" : "no", ev->result);
    }
}

static void press(gates_tree_t *t, gates_key_t key) {
    gates_key_event_t ev = { .key = key, .down = true };
    (void)gates_input_key(t, &ev);
    ev.down = false;
    (void)gates_input_key(t, &ev);
}

static void type(gates_tree_t *t, const char *s) {
    for (; *s != 0; s++) (void)gates_input_char(t, (gates_u32)(unsigned char)*s);
}

int main(void) {
    gates_tree_t *t = nullptr;
```

```
    if (!gates_is_ok(gates_tree_create(&(gates_tree_desc_t){0}, &t))) return 1;
    const gates_column_desc_t cols[] = {
        { .id = COL_NAME, .label = GATES_STR_INIT("Item"), .width = 120, .editable = true },
        { .id = COL_DONE, .label = GATES_STR_INIT("Done"), .width = 60, .kind = GATES_CELL_CHECK, .editable =
true },
    };
    gates_rows_model_t model = { .user = &list, .count = l_count, .id_at = l_id_at, .index_of = l_index_of,
                                 .cell = l_cell, .set_cell = l_set_cell };
    gates_node_t table;
    if (!gates_is_ok(gates_view_create(t, gates_tree_root(t),
                                       &(gates_view_desc_t){ .columns = cols, .column_count = 2, .header = true },
                                       &table)) ||
        !gates_is_ok(gates_view_set_model(t, table, &model)) ||
        !gates_is_ok(gates_widget_set_handler(t, table, on_view, nullptr)) ||
        !gates_is_ok(gates_layout_run(t, (gates_size_t){ 240, 160 }, gates_text_backend_builtin()))) {
        return 1;
    }
    /* A person selects the second row, presses F2, retypes the name and presses Enter. */
    gates_tree_set_focus(t, table);
    press(t, GATES_KEY_DOWN);
    press(t, GATES_KEY_DOWN);
    press(t, GATES_KEY_F2);
    type(t, "Bread");
    press(t, GATES_KEY_ENTER);
    /* Space toggles the row's check column. */
    press(t, GATES_KEY_SPACE);
    (void)gates_tree_dispatch_events(t, 0);

    /* An empty name is refused: the editor stays open, marked invalid, until Escape. */
    press(t, GATES_KEY_F2);
    press(t, GATES_KEY_DELETE);
    press(t, GATES_KEY_ENTER);
    printf("refused empty name: editor %s\n", gates_view_editing(t, table, nullptr, nullptr) ? "still open" :
"closed");
    press(t, GATES_KEY_ESCAPE);
    gates_tree_destroy(t);
    return 0;
}
```

5.4 여러 줄을 한꺼번에: 다중 선택

`multi_select` 로 만든 뷰는 여러 줄을 고르게 한다. Shift+클릭과 Shift+화살표는 기준점(anchor)에서 범위를 만들고, Ctrl+클릭과 Ctrl+Space 는 한 줄을 넣거나 빼고, Ctrl+A 는 전부를 고르고, Ctrl+화살표는 선택은 그대로 두고 포커스만 옮긴다. 줄을 누른 채 끌면 지나간 줄이 골라지고(누를 때 Ctrl 을 쥐고 있었으면 더해지고), 줄 목록의 위나 아래를 벗어나면 포인터가 움직일 때마다 한 줄씩 스크롤된다(0.10.0). 선택은 줄과 마찬가지로 프로그램의 것이다. 모델은 `next_selected(row)`(`row` 이후의 첫 선택 줄)에 답하고, 몸짓마다 GATES_EVENT_SELECT_REQUESTED 가 온다(`ev->result` 는 ONE, TOGGLE, RANGE, ADD_RANGE, ALL 가운데 하나, `ev->item` 은 대상 줄, `ev->anchor` 는 범위가 시작하는 줄). 프로그램은 선택을 바꾸고 `gates_view_model_changed` 를 부른다.

gates 는 그리거나 설명하거나 복사하는 줄만 묻기 때문에, 프로그램은 선택한 줄이 몇 개든 작은 모양으로 둘 수 있다. 범위 몇 개로 두면 백만 줄 "전부" 도 범위 하나다. gates 에는 그런 저장소가 있다(0.10.0). `gates_selection_create` 가 하나를 만들고, 모델의 `next_selected` 는 `gates_selection_next` 로 답하며, `gates_view_apply_selection(tree, ev, sel)` 이 요청을 적용하고(id 는 모델을 거쳐 줄 번호가 된다) 다시 그린다. 줄을 넣거나 지우면 `gates_selection_rows_inserted` / `_rows_removed` 가 범위를 함께 옮기고, `gates_selection_range` 로 범위를 하나씩 훑을 수 있다. 선택을 따로 다루는 프로그램은 제 방식대로 두면 된다. 뷰는 저장소를 읽지 않는다. Ctrl+C 는 선택한 줄을 `GATES_VIEW_COPY_MAX`(10000)개까지 스스로 한 줄씩 복사하고, 그보다 많으면 GATES_EVENT_COPY_REQUESTED 를 보내 복사를 프로그램에 맡긴다. `gates_view_selected`

는 계속 포커스 줄이다. 화면 낭독기에는 여럿을 고르는 목록으로 보인다(gates_access_set_item_selected 가 한 줄을 넣거나 빼며, 이것도 요청이다).

```
/* manual example (host): a million rows, selected together - kept in a selection store.
 * expect: 0..999999 after Shift+End; 1..999999 after Ctrl+Space on row 0; 0..999999 after Ctrl+A; copy asked for
1000000 rows */
#include <gates/gates.h>

#include <stdio.h>

#define ROWS 1000000u

/* The model: row r has id r + 1; the store answers next_selected. */
typedef struct app_t {
    gates_selection_t *sel;
    gates_node_t view;
    int copy_asked;
} app_t;

static gates_u64 m_count(void *u) { (void)u; return ROWS; }
static gates_item_id_t m_id_at(void *u, gates_u64 row) { (void)u; return row < ROWS ? row + 1 : 0; }
static bool m_index_of(void *u, gates_item_id_t id, gates_u64 *row) {
    (void)u;
    if (id == 0 || id > ROWS) return false;
    *row = id - 1;
    return true;
}
static gates_err_t m_cell(void *u, gates_item_id_t id, gates_column_id_t col, gates_cell_t *out) {
    (void)u; (void)id; (void)col;
    out->text = GATES_STR("a row");
    return GATES_OK;
}
static gates_u64 m_next_selected(void *u, gates_u64 row) { return gates_selection_next(((app_t *)u)->sel, row); }

/* A person's gesture is a request: the store applies it, then the view repaints. */
static void on_view(gates_tree_t *t, const gates_event_t *ev, void *user) {
    app_t *a = user;
    if (ev->kind == GATES_EVENT_COPY_REQUESTED) a->copy_asked++;
    if (ev->kind == GATES_EVENT_SELECT_REQUESTED) (void)gates_view_apply_selection(t, ev, a->sel);
}

static void press(gates_tree_t *t, gates_key_t key, bool ctrl, bool shift) {
    gates_key_event_t ev = { .key = key, .ctrl = ctrl, .shift = shift, .down = true };
    (void)gates_input_key(t, &ev);
    (void)gates_tree_dispatch_events(t, 0);
}

static void show(const app_t *a, const char *after, bool last) {
    gates_u64 lo = 0, hi = 0;
    for (gates_u32 i = 0; gates_selection_range(a->sel, i, &lo, &hi); i++) {
        printf("%s%llu..%llu", i > 0 ? "," : "", (unsigned long long)lo, (unsigned long long)hi);
    }
    printf(" after %s%s", after, last ? "" : "; ");
}

static gates_err_t no_clip_get(void *ctx, gates_allocator_t al, gates_u8 **out, gates_usize_t *n) {
    (void)ctx; (void)al;
    *out = nullptr;
    *n = 0;
    return GATES_OK;
}
static gates_err_t no_clip_set(void *ctx, gates_str_t text) { (void)ctx; (void)text; return GATES_OK; }
```

```
int main(void) {
    gates_tree_t *t = nullptr;
    static app_t a;
    if (!gates_is_ok(gates_tree_create(&(gates_tree_desc_t){0}, &t)) ||
        !gates_is_ok(gates_selection_create((gates_allocator_t){0}, &a.sel))) {
        return 1;
    }
    gates_rows_model_t model = { .user = &a, .count = m_count, .id_at = m_id_at, .index_of = m_index_of,
                                 .cell = m_cell, .next_selected = m_next_selected };
    if (!gates_is_ok(gates_view_create(t, gates_tree_root(t), &(gates_view_desc_t){ .multi_select = true },
&a.view)) ||
        !gates_is_ok(gates_view_set_model(t, a.view, &model)) ||
        !gates_is_ok(gates_widget_set_handler(t, a.view, on_view, &a)) ||
        !gates_is_ok(gates_layout_run(t, (gates_size_t){ 300, 200 }, gates_text_backend_builtin()))) {
        return 1;
    }
    gates_tree_set_clipboard(t, &(gates_clipboard_t){ .get_text = no_clip_get, .set_text = no_clip_set });
    gates_tree_set_focus(t, a.view);
    press(t, GATES_KEY_DOWN, false, false);  /* row 0 alone */
    press(t, GATES_KEY_END, false, true);    /* Shift+End: 0 to the last row */
    show(&a, "Shift+End", false);
    press(t, GATES_KEY_HOME, true, false);   /* Ctrl+Home: the focus moves, nothing asked */
    press(t, GATES_KEY_SPACE, true, false);  /* Ctrl+Space: row 0 leaves the selection */
    show(&a, "Ctrl+Space on row 0", false);
    press(t, GATES_KEY_A, true, false);      /* Ctrl+A: all, one range */
    show(&a, "Ctrl+A", false);
    press(t, GATES_KEY_C, true, false);      /* more than GATES_VIEW_COPY_MAX rows: the program is asked */
    printf("copy asked for %u rows\n", a.copy_asked == 1 ? ROWS : 0u);
    gates_tree_destroy(t);
    gates_selection_destroy(a.sel);
    return 0;
}
```

5.5 한 레코드의 필드: 속성 격자

한 레코드에 형식 있는 필드가 많을 때(문서 설정, 검사기에서 고른 개체) `gates_propgrid_create` (gates/propgrid.h)는 이름과 편집기를 한 줄씩 늘어놓는다. `gates_propgrid_add_text`, `_bool`, `_choice`, `_number` 는 글상자, 체크 상자, 선택, 스핀 상자를 만든다. 속성은 분류별로 묶이고 분류마다 접을 수 있는 그룹 상자가 되며, 분류 없는 속성이 맨 앞에 온다. 속성마다 바뀌지 않는 id 가 있다. `gates_propgrid_editor` 는 그 편집기를 주어 편집기 자신의 함수로 값을 쓰고 읽게 하고, 처리기 하나 (`gates_propgrid_set_handler`)가 사람이 바꾼 것을 모두 VALUE_CHANGED 로 듣는다. 속성 id 는 `ev->result` 에, 값은 `ev->text`, `ev->checked`, `ev->value` 에 담긴다.

```
/* manual example (host): a property grid over a document's settings.
 * expect: 4 properties in 2 categories; property 2 (Pages) is now 13; property 3 (Draft) is now off */
#include <gates/gates.h>

#include <stdio.h>

enum { P_TITLE = 1, P_PAGES, P_DRAFT, P_PAPER };

/* One handler hears every change, by property id. */
static void on_property(gates_tree_t *tree, const gates_event_t *ev, void *user) {
    (void)tree;
    (void)user;
    if (ev->result == P_PAGES) printf("property %u (Pages) is now %lld; ", ev->result, (long long)ev->value);
    if (ev->result == P_DRAFT) printf("property %u (Draft) is now %s\n", ev->result, ev->checked ? "on" : "off");
}

int main(void) {
```

```
    gates_tree_t *t = nullptr;
    if (!gates_is_ok(gates_tree_create(&(gates_tree_desc_t){0}, &t))) return 1;
    gates_node_t grid;
    const gates_option_t papers[] = { { .id = 1, .label = GATES_STR_INIT("A4") }, { .id = 2, .label =
GATES_STR_INIT("Letter") } };
    gates_range_t pages = { .min = 1, .max = 999, .value = 12 };
    if (!gates_is_ok(gates_propgrid_create(t, gates_tree_root(t), &grid)) ||
        !gates_is_ok(gates_propgrid_add_text(t, grid, GATES_STR("Document"), P_TITLE, GATES_STR("Title"),
GATES_STR("Notes"))) ||
        !gates_is_ok(gates_propgrid_add_number(t, grid, GATES_STR("Document"), P_PAGES, GATES_STR("Pages"),
&pages)) ||
        !gates_is_ok(gates_propgrid_add_bool(t, grid, GATES_STR("Print"), P_DRAFT, GATES_STR("Draft"), true)) ||
        !gates_is_ok(gates_propgrid_add_choice(t, grid, GATES_STR("Print"), P_PAPER, GATES_STR("Paper"), papers,
2, 1)) ||
        !gates_is_ok(gates_propgrid_set_handler(t, grid, on_property, nullptr)) ||
        !gates_is_ok(gates_layout_run(t, (gates_size_t){ 320, 320 }, gates_text_backend_builtin()))) {
        return 1;
    }
    printf("%u properties in %d categories; ", gates_propgrid_count(t, grid),
           !gates_node_eq(gates_propgrid_category(t, grid, GATES_STR("Print")), GATES_NODE_NULL) ? 2 : 1);

    /* A person steps Pages up in its spin box, then turns Draft off with Space. */
    gates_tree_set_focus(t, gates_node_first_child(t, gates_propgrid_editor(t, grid, P_PAGES)));
    gates_key_event_t up = { .key = GATES_KEY_UP, .down = true };
    (void)gates_input_key(t, &up);
    gates_tree_set_focus(t, gates_propgrid_editor(t, grid, P_DRAFT));
    gates_key_event_t space = { .key = GATES_KEY_SPACE, .down = true };
    (void)gates_input_key(t, &space);
    space.down = false;
    (void)gates_input_key(t, &space);
    (void)gates_tree_dispatch_events(t, 0);
    gates_tree_destroy(t);
    return 0;
}
```

5.6 모델의 규칙

모델 콜백은 UI 스레드에서 돌고, 이벤트 전달 중이나 다른 콜백 안에서는 돌지 않으며, `gates_view_set_model(view, nullptr)` 이 돌아온 뒤에는 불리지 않는다. 칸의 글은 모델을 다음에 부를 때까지 빌린 것이다. 모델은 `gates_view_model_changed` 로 알리지 않고 그 사이에 바뀌어서는 안 된다. `set_cell` 만은 모델을 바꿔도 되는 콜백이다(뷰가 그 뒤에 다시 읽는다).

6 글자 입력

이 장의 차례

6.1 텍스트 상자 26
6.2 한도는 질문이다 26
6.3 설치된 입력기 27
6.4 0.11.0 의 유니코드 한계 27
6.5 클립보드 27
6.6 바깥의 다른 인코딩 28
6.7 여러 줄: 편집기 29
6.8 큰 글 32

헤더: `gates/widget.h`(텍스트 상자), `gates/text_edit.h`, `gates/clipboard.h`, `gates/input.h`, `gates/ui.h`.

6.1 텍스트 상자

텍스트 상자는 UTF-8 글 한 줄을 편집한다. 사람은 입력하고, Shift 와 포인터로 고르고(Shift+클릭은 넓히고, 두 번 누르면 단어, 세 번 누르면 전부), 글자 단위와 처음·끝(Home, End)으로 옮기고, 복사·잘라내기·붙여넣기(Ctrl+C, Ctrl+X, Ctrl+V)와 되돌리기·다시 하기(Ctrl+Z, Ctrl+Y)를 한다. 편집이 성공할 때마다 확정된 글과 함께 TEXT_CHANGED 가 대기열에 들어간다. 프로그램이 부르는 `gates_textbox_set_text` 는 조용하고 되돌리기 이력을 지운다. 모든 텍스트 상자 밑에는 편집 코어(`gates/text_edit.h`)가 있다. 글, 캐럿, 고른 범위, 열린 조합을 UTF-8 바이트 위치로 다룬다.

설정	효과
`gates_textbox_set_read_only`	포커스, 고르기, 복사는 된다. 편집, 붙여넣기, 잘라내기, 되돌리기, 조합은 거절한다
`gates_textbox_set_password`	글자마다 * 하나, 복사·잘라내기 없음, 이벤트에 글 없음, 되돌리기 없음, IME 없음
`gates_textbox_set_max_bytes`	UTF-8 바이트로 잰 최대 길이(아래를 보라)
`gates_textbox_set_undo_limits`	되돌리기 이력의 한도. 기본 64개, 16384 바이트
`gates_textbox_set_invalid`	틀림 모양(폼이 대신 해 준다)

6.2 한도는 질문이다

입력이 최대 길이를 넘으려 하면 gates 는 아무것도 넣지 않고 묻는다. LIMIT_EXCEEDED 이벤트에 거절된 입력과, 그 가운데 몇 바이트가 들어갈 수 있는지가 담긴다. 프로그램은 사람에게 들어가는 만큼 넣을지, 버릴지 묻고, `gates_textbox_accept_fit` 이나 `gates_textbox_discard_rejected` 로 답한다. 붙여 넣은 계좌 번호를 소리 없이 반으로 자르는 실패를 이렇게 피한다.

```
/* manual example (host): a text box with a limit - the overflow is a question.
 * expect: offered 7 bytes, 5 fit; kept "Seoul" */
#include <gates/gates.h>

#include <stdio.h>

static gates_u32 offered, fit;

static void on_box(gates_tree_t *tree, const gates_event_t *ev, void *user) {
    (void)tree;
```

```
    (void)user;
    if (ev->kind == GATES_EVENT_LIMIT_EXCEEDED) {
        offered = (gates_u32)ev->text.size;
        fit = ev->fit_bytes;
    }
}

int main(void) {
    gates_tree_t *t = nullptr;
    if (!gates_is_ok(gates_tree_create(&(gates_tree_desc_t){0}, &t))) return 1;
    gates_node_t city;
    if (!gates_is_ok(gates_textbox_create(t, gates_tree_root(t), GATES_STR(""), 20, &city)) ||
        !gates_is_ok(gates_textbox_set_max_bytes(t, city, 5)) ||
        !gates_is_ok(gates_widget_set_handler(t, city, on_box, nullptr))) {
        return 1;
    }
    gates_tree_set_focus(t, city);
    /* Typed or pasted input (an IME delivers its result the same way). */
    (void)gates_input_commit(t, GATES_STR("Seoul!!"));
    (void)gates_tree_dispatch_events(t, 0);
    /* Nothing was inserted: the box holds the input as an offer. Here the
     * answer is "keep what fits"; gates_textbox_discard_rejected drops it. */
    if (!gates_is_ok(gates_textbox_accept_fit(t, city))) return 1;
    gates_str_t text = gates_textbox_text(t, city);
    printf("offered %u bytes, %u fit; kept \"%.*s\"\n", offered, fit, (int)text.size, (const char *)text.ptr);
    gates_tree_destroy(t);
    return 0;
}
```

6.3 설치된 입력기

gates 는 입력기를 구현하지 않는다. Windows 에서는 설치된 IME(한국어, 일본어, 중국어 …)가 IMM32 를 통해 모든 텍스트 상자에서 동작한다. 조합 중인 음절은 캐럿 자리에 바로 보이고, 후보 창은 캐럿 아래에 열리며, 끝난 글은 편집 하나로 들어온다. 읽기 전용이나 비밀번호 상자는 포커스를 가진 동안 IME 를 끈다. 글을 스스로 넣는 프로그램(시험, 자동화)을 위해 `gates_input_commit` 은 IME 처럼 글을 넣고, `gates_input_preedit` 은 조합을 보인다.

6.4 0.11.0 의 유니코드 한계

- 글은 어디서나 UTF-8 이다. 잘못된 입력은 거절하며, 몰래 고치지 않는다. 다른 인코딩의 글은 가장자리에서 바꾼다(아래).
- 글자마다 글꼴에서 가져온 제 폭(advance)이 있다. UI 글꼴에서는 비례폭, 고정폭 글꼴에서는 고정폭이다(8장). 글줄의 폭은 글자 폭의 합과 정확히 같으므로 캐럿, 고른 범위, 누른 자리는 늘 글자 사이에 떨어진다.
- 커닝, 합자, 셰이핑은 없다. 셰이핑이 필요한 문자(아랍 문자, 인도계 문자, 타이 문자)는 글자가 하나씩 따로 보이고, 오른쪽에서 왼쪽으로 쓰는 글은 다시 배열하지 않는다.
- 편집은 문자소 묶음(grapheme cluster)이 아니라 코드 포인트 단위로 움직인다. 바탕 글자와 결합 부호는 두 걸음이다.
- Win32 백엔드는 시스템의 진짜 글꼴로 그리며, 그 글꼴에 없는 글자(Segoe UI 의 한글)는 시스템의 대체 글꼴에서 가져온다. 내장 백엔드(시험, 창 없음)는 고정폭이고 ASCII 를 그리며, 나머지는 맞는 너비의 상자로 그린다.
- 텍스트 상자는 한 줄이다. 여러 줄은 편집기의 몫이다(아래).

6.5 클립보드

창이 플랫폼 클립보드를 대 준다(`gates/clipboard.h`). 글은 UTF-8 로 넘나들고 가장자리에서 바뀐다. Windows 에서는 클립보드의 줄 끝이 CR LF 다. 복사는 LF 를 CR LF 로 쓰고 붙여넣기는 CR LF 를 LF 로 읽으므로,

다른 프로그램을 거쳐도 줄 끝이 그대로다(0.15.1). 클립보드 제공자가 없는 트리(창 없음)에서는 복사, 잘라내기, 붙여넣기가 아무 일도 하지 않는다.

6.6 바깥의 다른 인코딩

프로그램 안에서는 API, 텍스트 상자, 편집기, 텍스트 버퍼 모두가 UTF-8 이다. 바깥은 늘 그렇지 않다. 파일은 바이트 순서 표시가 붙은 UTF-16 일 수 있고, 한국어 Windows 의 콘솔은 949 코드 페이지를 쓰며, 옛 프로그램은 EUC-KR 로 쓴다. `gates/encoding.h` 가 가장자리에서 바꿔 주므로, 프로그램은 안에서 인코딩 하나만 두고 환경마다 그 환경의 인코딩으로 만난다(0.10.0).

- `gates_encoding_to_utf8(enc, bytes, size, flags, alloc, &out, &out_size, &bad_at)` 는 글을 들여온다. 그 인코딩의 바이트 순서 표시는 건너뛴다. `gates_encoding_from_utf8` 은 다시 내보낸다(`enc.bom` 이면 표시를 앞에 붙인다). 결과는 `alloc`({0} = 힙)에서 받고, 세지 않는 0 바이트로 끝나며(그대로 C 문자열이나 와이드 문자열), 부른 쪽이 푼다.
- UTF-8, UTF-16, UTF-32 는 두 바이트 순서 모두 gates 가 어디서나 바꾼다. 코드 페이지는 Windows 번호로 고르며 (`GATES_CODEPAGE_CP949` 949, `GATES_CODEPAGE_EUC_KR` 51949, 932, 936, 1252 …, 0 = 시스템의 것) 플랫폼의 표가 있어야 한다. Win32 앱은 만들어질 때 변환기를 넣고, 변환기가 없으면 UNSUPPORTED 다. `gates_encoding_system_codepage` 와 `gates_encoding_console_codepage` 는 시스템과 콘솔이 쓰는 코드 페이지를 알려 준다.
- 깨진 입력은 U+FFFD 가 된다. `GATES_ENCODING_STRICT` 를 주면 대신 거절하고 `bad_at` 이 몇 번째 바이트인지 알려 준다. 코드 페이지에 없는 글자는 그 코드 페이지의 `?` 가 되며, strict 면 거절된다.
- `gates_encoding_detect` 는 어디서 왔는지 모르는 바이트를 짐작한다. 바이트 순서 표시가 먼저 정하고, 다음은 0 바이트로 보는 UTF-16, 그다음 올바른 UTF-8, 아니면 시스템 코드 페이지다. `gates_utf8_valid` 는 글을 편집기에 넣기 전에 확인한다 (편집기는 UTF-8 이 아닌 글을 거절한다).
- 물어볼 OS 가 없는 대상(마이크로컨트롤러, RTOS 의 GUI)에는 `gates_codepage_converter_cp949()` 가 있다. Windows 11 에서 측정한 표로 949(CP949)와 51949(EUC-KR)를 Windows 와 똑같이 바꾸는 gates 자체 변환기다(0.11.0). `gates_encoding_set_codepage_converter` 로 넣으며, 이것을 부르는 프로그램만 그 표(약 120 KB)를 싣는다.
- 메모리를 아껴야 하는 대상에는 할당 없이 쓴다. `gates_encoding_to_utf8_buf` / `_from_utf8_buf` 는 부른 쪽의 버퍼에 쓰고(버퍼 없이 부르면 크기만 알려 준다), `gates_encoding_stream_t` 는 조각으로 바꾼다. 블록으로 읽는 파일, 직렬선으로 오는 바이트처럼 조각 끝에서 잘린 글자는 다음 조각으로 넘긴다. 둘 다 UTF 형식을 다룬다.
- Windows 에서는 `main` 의 인자가 ANSI 코드 페이지로 오고, UTF-8 콘솔이 아니면 UTF-8 `printf` 가 깨져 보인다. `gates_args_utf8_win32` 는 명령줄을 UTF-8 인자로 주고(Windows 규칙으로 나누는 `gates_args_split` 은 어디서나 쓸 수 있다), `gates_console_write_win32` 는 콘솔에 UTF-8 을 쓰며 출력이 다른 곳으로 돌려졌으면 바이트를 그대로 쓴다.

```
/* manual example (host): text from outside in another encoding, UTF-8 inside, back out.
 * expect: UTF-16LE with a 2-byte mark; 9 characters in 13 UTF-8 bytes; out as UTF-32BE in 36 bytes; strict
refuses byte 2; code page 949 here: unsupported */
#include <gates/gates.h>
#include <proven/heap.h>

#include <stdio.h>

static const char *name(gates_encoding_kind_t k) {
    static const char *const names[] = { "UTF-8", "UTF-16LE", "UTF-16BE", "UTF-32LE", "UTF-32BE", "a code page" };
    return names[k];
}

int main(void) {
    /* A file's bytes: a UTF-16LE mark, then "Hi, " and two Hangul syllables U+D55C U+AE00, then "!". */
    static const unsigned char file[] = { 0xFF, 0xFE, 'H', 0, 'i', 0, ',', 0, ' ', 0, 0x5C, 0xD5, 0x00, 0xAE,
                                          '!', 0, '\r', 0, '\n', 0 };
```

```
    gates_encoding_t enc;
    gates_usize_t mark = gates_encoding_detect(file, sizeof file, &enc);
    printf("%s with a %u-byte mark; ", name(enc.kind), (unsigned)mark);

    /* In: one encoding inside the program from here on (the mark is skipped). */
    gates_u8 *text = nullptr;
    gates_usize_t size = 0;
    if (!gates_is_ok(gates_encoding_to_utf8(enc, file, sizeof file, 0, (gates_allocator_t){0}, &text, &size,
nullptr))) {
        return 1;
    }
    gates_str_t s = { .ptr = text, .size = size };
    unsigned chars = 0;
    for (gates_u32 at = 0; at < size; chars++) at += gates_text_decode(s, at, &(gates_u32){0});
    printf("%u characters in %u UTF-8 bytes; ", chars, (unsigned)size);

    /* Out: whatever the other side wants. */
    gates_u8 *wide = nullptr;
    gates_usize_t wide_size = 0;
    if (!gates_is_ok(gates_encoding_from_utf8((gates_encoding_t){ .kind = GATES_ENCODING_UTF32BE }, s, 0,
                                              (gates_allocator_t){0}, &wide, &wide_size, nullptr))) {
        return 1;
    }
    printf("out as UTF-32BE in %u bytes; ", (unsigned)wide_size);

    /* Broken input: replaced with U+FFFD by default, or refused with where it broke. */
    gates_usize_t bad_at = 0;
    gates_u8 *junk = nullptr;
    gates_err_t err = gates_encoding_to_utf8((gates_encoding_t){ .kind = GATES_ENCODING_UTF8 }, "ok\xFF", 3,
                                             GATES_ENCODING_STRICT, (gates_allocator_t){0}, &junk, &size,
&bad_at);
    printf("strict refuses byte %u; ", err == PROVEN_ERR_INVALID_ENCODING ? (unsigned)bad_at : 99u);

    /* Code pages (949 = Korean Windows, EUC-KR and more) need a converter: on Windows the app
     * installs the platform's; gates_codepage_converter_cp949() is a built-in one for 949 and
     * 51949. This program installs none, so the call says so. */
    err = gates_encoding_to_utf8((gates_encoding_t){ .kind = GATES_ENCODING_CODEPAGE, .codepage =
GATES_CODEPAGE_CP949 },
                                 "\xC7\xD1", 2, 0, (gates_allocator_t){0}, &junk, &size, nullptr);
    printf("code page 949 here: %s\n", err == PROVEN_ERR_UNSUPPORTED ? "unsupported" : "converted");

    gates_allocator_t heap = proven_heap_allocator();
    heap.free_fn(heap.ctx, text);
    heap.free_fn(heap.ctx, wide);
    return 0;
}
```

6.7 여러 줄: 편집기

`gates_editor_create`(gates/editor.h)는 텍스트 버퍼 위에 여러 줄 편집기를 만든다. 보이는 줄만 그리므로 긴 파일도 창만큼만 든다. 화살표, Home/End, PageUp/PageDown 과 그 Ctrl 조합이 캐럿을 옮기고(Shift 는 선택, Shift+클릭은 넓히고, 두 번 누르면 단어, 세 번 누르면 줄 끝까지 포함한 줄), Enter 는 글의 원래 줄 끝을 따르고, Tab 은 설명에서 탭 입력을 청하지 않는 한 포커스를 옮기며, Ctrl+A/C/X/V/Z/Y 는 어디서나처럼 동작한다. 사람이 고친 것은 되돌릴 수 있고(이어 치거나 지운 것은 한 걸음), `gates_editor_modified` 는 글이 설정하거나 저장한 것과 다른지 알려 준다. 프로그램은 `gates_editor_buffer` 로 글을 읽고 `gates_editor_set_text` 나 `gates_editor_replace` 로 바꾼다. TEXT_CHANGED 는 글을 담지 않으므로 긴 파일이 이벤트에 복사되는 일은 없다.

```c
/* manual example (host): a multi-line editor - typing, lines, undo and the modified mark.
 * expect: 3 lines, caret on line 2; after undo: 2 lines, modified: no */
#include <gates/gates.h>

#include <stdio.h>

static void press(gates_tree_t *t, gates_key_t key, bool ctrl) {
    gates_key_event_t ev = { .key = key, .ctrl = ctrl, .down = true };
    (void)gates_input_key(t, &ev);
    ev.down = false;
    (void)gates_input_key(t, &ev);
}

static void type(gates_tree_t *t, const char *s) {
    for (; *s != 0; s++) (void)gates_input_char(t, (gates_u32)(unsigned char)*s);
}

int main(void) {
    gates_tree_t *t = nullptr;
    if (!gates_is_ok(gates_tree_create(&(gates_tree_desc_t){0}, &t))) return 1;
    gates_node_t notes;
    gates_editor_desc_t desc = { .rows = 8, .cols = 40 };
    if (!gates_is_ok(gates_editor_create(t, gates_tree_root(t), &desc, &notes)) ||
        !gates_is_ok(gates_editor_set_text(t, notes, GATES_STR("Shopping\nMilk"))) ||
        !gates_is_ok(gates_node_set_access_name(t, notes, GATES_STR("Notes"))) ||
        !gates_is_ok(gates_layout_run(t, (gates_size_t){ 360, 200 }, gates_text_backend_builtin()))) {
        return 1;
    }
    /* A person goes to the end, starts a new line and types. */
    gates_tree_set_focus(t, notes);
    press(t, GATES_KEY_END, true);
    press(t, GATES_KEY_ENTER, false);
    type(t, "Bread");
    const gates_text_buffer_t *text = gates_editor_buffer(t, notes);
    gates_u32 caret = 0;
    gates_editor_selection(t, notes, nullptr, &caret);
    printf("%u lines, caret on line %u; ", gates_text_buffer_line_count(text), gates_text_buffer_line_of(text,
caret));

    /* Ctrl+Z takes the typing back, then the new line: the text is as it was set. */
    press(t, GATES_KEY_Z, true);
    press(t, GATES_KEY_Z, true);
    printf("after undo: %u lines, modified: %s\n", gates_text_buffer_line_count(text),
           gates_editor_modified(t, notes) ? "yes" : "no");
    gates_tree_destroy(t);
    return 0;
}
```

설명에서 줄 바꿈(행은 들어가는 마지막 공백 뒤에서 나뉘고, 그러면 Up 과 Down 은 행 단위로 움직이며, Home 과 End 는 줄보다 먼저 행의 처음과 끝으로 가고, 행 끝의 캐럿은 그 자리에 그려진다), 줄 번호 여백, 자동 들여쓰기를 켤 수 있다. Tab 이 탭을 칠 때 Tab 과 Shift+Tab 은 선택한 줄을 들여쓰고 내어쓰며, Ctrl+Tab 이 포커스를 다음으로 옮기므로 키보드가 갇히지 않는다. 강조는 프로그램의 몫이다. `gates_editor_set_styles` 가 스타일 바이트를 색에 잇고, 스타일러(`gates_editor_set_styler`)는 그리기 직전에 곧 보일 줄 가운데 스타일이 낡은 것을 칠하라는 요청을 받는다(고치면 그 줄이 다시 낡는다). 그래서 스타일 일은 글의 길이가 아니라 보이는 곳을 따른다. `gates_editor_find` 는 다음 일치를 선택하고, 표시 (`gates_editor_mark_add`)는 주변 글이 바뀌어도 자리를 지킨다. 입력기는 캐럿에서 조합하고 조합 중인 글은 밑줄로 그려지며, 결과는 한 번의 편집으로 들어온다. 화면 낭독기는 편집기의 글을 글자, 단어, 줄 단위로 읽고(줄은 보이는 대로, 곧 감긴 행이다 (`gates_access_text_line`); 문단은 글의 줄이다), 범위는 행마다 사각형 하나로 받는다 (`gates_access_text_rects`).

```
/* manual example (host): highlighting through a styler, and find.
 * expect: styled up to line 11 of 200; "TODO" found on line 150 */
#include <gates/gates.h>

#include <stdio.h>
#include <string.h>

enum { PLAIN, NUMBER, KEYWORD };

/* The program's highlighter: numbers and the word "TODO". It only styles the
 * range it is given - the editor asks for what is about to show. */
static gates_u32 styled_to;
static void highlight(void *user, gates_text_buffer_t *text, gates_u32 from, gates_u32 to) {
    (void)user;
    gates_text_buffer_set_style(text, from, to, PLAIN);
    for (gates_u32 i = from; i < to; i++) {
        gates_u8 c = gates_text_buffer_byte(text, i);
        if (c >= '0' && c <= '9') gates_text_buffer_set_style(text, i, i + 1, NUMBER);
    }
    gates_u32 at = from;
    while (gates_text_buffer_find(text, at, GATES_STR("TODO"), 0, &at) && at + 4 <= to) {
        gates_text_buffer_set_style(text, at, at + 4, KEYWORD);
        at += 4;
    }
    styled_to = to;
}

int main(void) {
    gates_tree_t *t = nullptr;
    if (!gates_is_ok(gates_tree_create(&(gates_tree_desc_t){0}, &t))) return 1;
    static char text[8000];
    int n = 0;
    for (int i = 1; i <= 200; i++) n += snprintf(text + n, sizeof text - (size_t)n, i == 150 ? "TODO %d\n" : "step
%d\n", i);
    gates_node_t ed;
    gates_editor_desc_t desc = { .rows = 10, .line_numbers = true, .wrap = true };
    const gates_editor_style_t styles[] = { {0}, { .token = GATES_COLOR_FOCUS_RING }, { .token =
GATES_COLOR_ERROR } };
    if (!gates_is_ok(gates_editor_create(t, gates_tree_root(t), &desc, &ed)) ||
        !gates_is_ok(gates_editor_set_text(t, ed, (gates_str_t){ (const gates_u8 *)text, (gates_usize_t)n })) ||
        !gates_is_ok(gates_editor_set_styles(t, ed, styles, 3)) ||
        !gates_is_ok(gates_editor_set_styler(t, ed, highlight, nullptr)) ||
        !gates_is_ok(gates_layout_run(t, (gates_size_t){ 320, 200 }, gates_text_backend_builtin()))) {
        return 1;
    }
    /* Painting styles only the lines shown (and the next one). */
    gates_draw_list_t dl;
    if (!gates_is_ok(gates_draw_list_init(&dl, (gates_allocator_t){0}, 0)) ||
        !gates_is_ok(gates_paint_tree(t, &dl, gates_theme_light(), gates_text_backend_builtin()))) {
        return 1;
    }
    const gates_text_buffer_t *buf = gates_editor_buffer(t, ed);
    printf("styled up to line %u of %u; ", gates_text_buffer_line_of(buf, styled_to),
gates_text_buffer_line_count(buf) - 1);

    /* Find selects the match and scrolls to it. */
    if (gates_editor_find(t, ed, GATES_STR("todo"), GATES_FIND_IGNORE_CASE, true)) {
        gates_u32 caret = 0;
        gates_editor_selection(t, ed, nullptr, &caret);
        printf("\"TODO\" found on line %u\n", gates_text_buffer_line_of(buf, caret) + 1);
    }
    gates_draw_list_deinit(&dl);
```

```
    gates_tree_destroy(t);
    return 0;
}
```

6.8 큰 글

`gates/text_buffer.h` 는 도구가 고치는 길이의 UTF-8 글(1 GiB 까지)을 담는다. 줄 시작 색인이 딸린 간격 버퍼, 바이트마다 스타일 바이트, 편집을 따라 움직이는 표시, 찾기가 있다. 읽기는 간격을 옮기지 않고 범위를 많아야 두 조각으로 돌려주므로, 프로그램은 다른 곳을 고치면서 보이는 줄을 읽을 수 있다. 여러 줄 편집기가 이 위에 서고, 프로그램이 큰 글을 불러오고 찾고 고칠 때 따로 쓸 수도 있다.

7 작업 스레드와 타이머

이 장의 차례

7.1 작업 스레드는 트리를 만지지 않는다 33
7.2 백그라운드 작업 35
7.3 타이머 36

헤더: `gates/post.h`, `gates/timer.h`.

7.1 작업 스레드는 트리를 만지지 않는다

트리는 UI 스레드의 것이다. 작업 스레드(파일 읽기, 장치와 이야기하기, 계산)는 결과를 메시지로 보내서 넘긴다. 프로그램은 작업 스레드를 시작하기 전에 UI 스레드에서 응용의 송신자(sender, `gates_app_sender`)를 받고 대상을 만든다. 대상은 트리와 결과를 받을 노드다(`gates_target`). 작업 스레드는 종류와 짐(payload)을 담은 메시지를 보낸다. UI 스레드는 안전한 시점에 그 노드의 메시지 처리기(`gates_node_set_message_handler`)에 전달하고 짐을 놓아 준다.

```
/* manual example (windows): a worker thread feeds a log through the sender.
 * The worker never touches the tree; it posts lines, and the UI thread
 * delivers them to the log's message handler. */
#include <gates/gates.h>

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <windows.h>

enum { MSG_LINE = 1 };

typedef struct job_t {
    gates_sender_t *sender;      /* retained for the worker */
    gates_target_t target;       /* built on the UI thread */
} job_t;

static void release_line(void *payload, void *ctx) {
    (void)ctx;
    free(payload); /* may run on any thread: only frees */
}

static DWORD WINAPI worker(void *arg) {
    job_t *job = arg;
    for (int i = 1; i <= 1000; i++) {
        char *line = malloc(40);
        if (line == nullptr) break;
        int n = snprintf(line, 40, "line %d from the worker", i);
        gates_message_t msg = { .target = job->target, .kind = MSG_LINE, .payload = line,
                                .bytes = (gates_usize_t)n, .release = release_line };
        gates_err_t err = gates_sender_post(job->sender, &msg);
        if (err == GATES_POST_FULL) { free(line); Sleep(5); i--; continue; } /* the caller keeps it */
        if (err == GATES_POST_CLOSED) { free(line); break; }                 /* the app is closing */
        if (!gates_is_ok(err)) { free(line); break; }
    }
    gates_sender_release(job->sender);
    return 0;
```

```
}

static void on_line(gates_tree_t *tree, gates_node_t node, gates_u32 kind, void *payload,
                    gates_usize_t bytes, void *user) {
    (void)user;
    if (kind == MSG_LINE) (void)gates_log_append(tree, node, (gates_str_t){ .ptr = payload, .size = bytes });
}

int main(void) {
    gates_app_t *app = nullptr;
    if (!gates_is_ok(gates_app_create(&(gates_app_desc_t){0}, &app))) return 1;
    gates_window_t *win = nullptr;
    gates_window_desc_t desc = { .title = GATES_STR("Worker"), .size = { 420, 300 } };
    if (!gates_is_ok(gates_window_create(app, &desc, nullptr, &win))) {
        gates_app_destroy(app);
        return 1;
    }
    gates_tree_t *t = gates_window_tree(win);
    gates_node_t root = gates_tree_root(t), log;
    job_t job = {0};
    gates_err_t err = gates_layout_set(t, root, GATES_LAYOUT_KIND_COLUMN);
    if (gates_is_ok(err)) err = gates_log_create(t, root, &(gates_log_desc_t){ .max_lines = 500 }, &log);
    if (gates_is_ok(err)) err = gates_layout_set_child_grow(t, log, 1);
    if (gates_is_ok(err)) err = gates_node_set_access_name(t, log, GATES_STR("Worker output"));
    if (gates_is_ok(err)) err = gates_node_set_message_handler(t, log, on_line, nullptr);
    if (gates_is_ok(err)) err = gates_app_sender(app, &job.sender);
    HANDLE thread = nullptr;
    if (gates_is_ok(err)) {
        job.target = gates_target(t, log);
        thread = CreateThread(nullptr, 0, worker, &job, 0, nullptr);
        if (thread == nullptr) gates_sender_release(job.sender);
    }
    if (gates_is_ok(err)) err = gates_app_run(app);
    gates_window_destroy(win);
    gates_app_destroy(app); /* closes the sender: the worker sees CLOSED and stops */
    if (thread != nullptr) {
        WaitForSingleObject(thread, INFINITE);
        CloseHandle(thread);
    }
    return gates_is_ok(err) ? 0 : 1;
}
```

이것이 안전한 까닭은 다음 규칙이다.

- 보내기는 막히지도 할당하지도 않는다. 대기열이 차면 `GATES_POST_FULL` 이 온다. 작업 스레드는 짐을 그대로 갖고 나중에 다시 하거나, 버리거나, 다음 것에 합친다. 대기열이 닫혔으면 `GATES_POST_CLOSED` 가 온다. 응용이 끝나는 중이니 멈추고 송신자를 놓는다.
- 보내기가 성공하면 짐은 gates 의 것이 되고 정확히 한 번 놓인다. 전달한 뒤, 대상 노드가 없어졌을 때, 더 새 스냅숏이 대신할 때(`replaceable`), 또는 끝날 때다. 보내기가 실패하면 짐은 여전히 작업 스레드의 것이다.
- 놓아 주는 함수는 어느 스레드에서든 돌 수 있다. 그 함수는 풀기만 한다.
- 대기열에는 한도가 있고(기본 메시지 1024 개, 짐 1 MiB, `gates_app_desc_t`), 한 차례에 많아야 64 개만 전달하므로 메시지가 쏟아져도 입력과 그리기가 굶지 않는다. 대기열은 256 칸으로 시작해 찰 때마다 UI 스레드에서 한도까지 늘어나므로, 한도를 크게 잡아도 몰릴 때만 메모리를 쓴다. 한도는 견딜 수 있는 지연으로 고른다. 대기열은 대략 한도 / 전달 속도만큼 기다리게 한다(초당 약 55000 개일 때 1024 는 20 ms 미만, 65536 은 1초가 넘는다). 한도를 아예 없애면 쏟아짐이 몇 분의 지연과 끝없는 메모리로 바뀔 뿐이다. 처리량은 대기열이 아니라 전달 속도가 정한다.
- 송신자는 마지막 참조가 놓일 때까지, 응용이 사라진 뒤에도, 모든 스레드에서 유효하다.

7.2 백그라운드 작업

진행률, 결과, 취소 단추가 있는 흔한 경우에는 gates_task_start(gates/task.h)가 보내는 일을 대신한다. 작업 함수는 자기 스레드에서 돌고(창이 스레드를 준다), gates_task_report 로 천분율과 짧은 글을 알리고, 이따금 gates_task_cancelled 를 보고, 결과를 돌려준다. UI 스레드에서 on_progress 는 가장 최근 보고를 보여 주고(아직 보이지 않은 보고는 쌓이지 않고 바뀐다), on_done 은 결과와 취소 여부를 담아 한 번 온다. 스레드는 대신 합쳐 준다(join). 창을 닫으면 돌고 있는 작업을 취소하고 작업이 돌아올 때까지 기다리므로, 작업 함수는 gates_task_cancelled 를 자주 봐야 한다.

```
/* manual example (windows): a background task counts primes while the window
 * shows its progress and a Cancel button stops it. The work never touches the
 * tree: it reports, polls for cancellation and returns; the UI hears both. */
#include <gates/gates.h>

#include <stdio.h>

#define LIMIT 3000000u

typedef struct count_t {
    gates_u32 primes;            /* written by the work, read in on_done */
    gates_node_t bar, status, cancel;
    gates_task_t *task;
} count_t;

static bool is_prime(gates_u32 n) {
    if (n < 2) return false;
    for (gates_u32 d = 2; d * d <= n; d++) {
        if (n % d == 0) return false;
    }
    return true;
}

/* On the worker thread. */
static gates_err_t count_primes(gates_task_t *task, void *user) {
    count_t *c = user;
    for (gates_u32 n = 0; n < LIMIT; n++) {
        if (is_prime(n)) c->primes++;
        if (n % (LIMIT / 100) == 0) {
            if (gates_task_cancelled(task)) return PROVEN_ERR_EOF;
            char text[48];
            int len = snprintf(text, sizeof text, "%u primes so far", c->primes);
            (void)gates_task_report(task, (gates_u32)((gates_u64)n * 1000 / LIMIT),
                                    (gates_str_t){ .ptr = (const gates_u8 *)text, .size = (gates_usize_t)len });
        }
    }
    return GATES_OK;
}

/* On the UI thread. */
static void on_progress(gates_tree_t *tree, gates_task_t *task, gates_u32 permille, gates_str_t text, void *user)
{
    (void)task;
    count_t *c = user;
    (void)gates_progress_set_value(tree, c->bar, (gates_i32)permille);
    (void)gates_widget_set_text(tree, c->status, text);
}

static void on_done(gates_tree_t *tree, gates_task_t *task, gates_err_t result, bool cancelled, void *user) {
    (void)task;
    count_t *c = user;
    char text[64];
```

```
    int len = cancelled ? snprintf(text, sizeof text, "Cancelled after %u primes", c->primes)
              : gates_is_ok(result) ? snprintf(text, sizeof text, "%u primes below %u", c->primes, LIMIT)
                                    : snprintf(text, sizeof text, "Stopped");
    (void)gates_widget_set_text(tree, c->status, (gates_str_t){ .ptr = (const gates_u8 *)text, .size =
(gates_usize_t)len });
    (void)gates_widget_set_disabled(tree, c->cancel, true);
    c->task = nullptr;
}

static void on_cancel(gates_tree_t *tree, gates_node_t node, void *user) {
    (void)tree;
    (void)node;
    count_t *c = user;
    if (c->task != nullptr) gates_task_cancel(c->task);
}

int main(void) {
    gates_app_t *app = nullptr;
    if (!gates_is_ok(gates_app_create(&(gates_app_desc_t){0}, &app))) return 1;
    gates_window_t *win = nullptr;
    gates_window_desc_t desc = { .title = GATES_STR("Primes"), .size = { 380, 160 } };
    if (!gates_is_ok(gates_window_create(app, &desc, nullptr, &win))) {
        gates_app_destroy(app);
        return 1;
    }
    gates_tree_t *t = gates_window_tree(win);
    gates_node_t root = gates_tree_root(t);
    count_t c = {0};
    gates_err_t err = gates_layout_set(t, root, GATES_LAYOUT_KIND_COLUMN);
    if (gates_is_ok(err)) err = gates_layout_set_padding(t, root, 12);
    if (gates_is_ok(err)) err = gates_layout_set_gap(t, root, 8);
    if (gates_is_ok(err)) err = gates_progress_create(t, root, 0, &c.bar);
    if (gates_is_ok(err)) err = gates_node_set_access_name(t, c.bar, GATES_STR("Progress"));
    if (gates_is_ok(err)) err = gates_label_create(t, root, GATES_STR("Starting..."), &c.status);
    if (gates_is_ok(err)) err = gates_button_create(t, root, GATES_STR("&Cancel"), on_cancel, &c, &c.cancel);
    gates_task_desc_t task = { .work = count_primes, .work_user = &c, .on_progress = on_progress,
                               .on_done = on_done, .ui_user = &c };
    if (gates_is_ok(err)) err = gates_task_start(t, &task, &c.task);
    if (gates_is_ok(err)) err = gates_app_run(app);
    gates_window_destroy(win); /* a task still running is cancelled and waited for */
    gates_app_destroy(app);
    return gates_is_ok(err) ? 0 : 1;
}
```

7.3 타이머

타이머는 노드에 속하며, 정한 간격 뒤에 UI 스레드에서 콜백을 한 번 또는 되풀이해 부른다. 취소는 즉시다. 취소한 타이머는 다시 울리지 않고, 노드나 트리를 없애면 취소된다. 늦은 되풀이 타이머는 한 번 울리고 지금부터 다시 잡힌다. 몰아서 울리지 않는다. 한가한 창은 플랫폼 타이머를 하나도 돌리지 않는다.

창의 트리에는 시계가 있다. 맨 트리(시험)는 프로그램에게서 시계를 받고, 시간이 언제 흐를지는 프로그램이 정한다.

```
/* manual example (host): a repeating timer, driven by a test clock.
 * expect: 3 ticks, then cancelled */
#include <gates/gates.h>

#include <stdio.h>

static gates_u64 now_ms;
static gates_u64 clock_now(void *ctx) { (void)ctx; return now_ms; }
```

```
static void on_tick(gates_tree_t *tree, gates_node_t node, gates_timer_id_t id, void *user) {
    (void)node;
    int *ticks = user;
    if (++*ticks == 3) (void)gates_timer_cancel(tree, id); /* a callback may cancel itself */
}

int main(void) {
    gates_tree_t *t = nullptr;
    if (!gates_is_ok(gates_tree_create(&(gates_tree_desc_t){0}, &t))) return 1;
    /* A window's tree has a clock; a bare tree gets one from the program. */
    gates_tree_set_clock(t, clock_now, nullptr, nullptr);
    int ticks = 0;
    gates_timer_id_t id;
    if (!gates_is_ok(gates_timer_start(t, gates_tree_root(t), 100, true, on_tick, &ticks, &id))) return 1;
    for (int step = 0; step < 10; step++) {
        now_ms += 100;
        (void)gates_tree_run_timers(t); /* the window does this when its timer fires */
    }
    printf("%d ticks, then %s\n", ticks, gates_timer_active(t, id) ? "still running" : "cancelled");
    gates_tree_destroy(t);
    return 0;
}
```

움직임을 보이기 전에 `gates_window_reduced_motion` 을 묻는다. 사람이 애니메이션 효과를 꺼 두었으면 끝 상태를 바로 보인다. app_settings 의 "applying" 단계가 그렇게 한다. 타이머가 진행 막대를 채우거나, 막대가 곧바로 가득 찬다.

8 테마, 단위, 배율

이 장의 차례

8.1 논리 단위 38
8.2 테마 38
8.3 글꼴 39
8.4 그리기와 글자 백엔드 39

헤더: `gates/theme.h`, `gates/geometry.h`, `gates/draw.h`, `gates/render.h`, `gates/text.h`.

8.1 논리 단위

API 의 모든 좌표와 크기는 논리 단위, 1/96 인치다. 배치 값, 노드 사각형, 바라는 크기, 창 크기, 포인터 위치가 다 그렇다. 창은 가장자리에서 한 번 바꾼다. 그릴 때는 장치 픽셀로, 포인터를 알릴 때는 다시 단위로 바꾼다. 그래서 같은 프로그램이 100 % 화면과 200 % 화면에서 같은 실제 크기다. `gates_px` 와 `gates_logical` 이 그 변환이다(정수 단위에서는 정확한 역).

```
/* manual example (host): logical units and themes.
 * expect: 100 units = 150 px at 144 dpi; 150 px = 100 units; light theme: 0 issues */
#include <gates/gates.h>

#include <stdio.h>

int main(void) {
    /* Every coordinate in the API is a logical unit, 1/96 inch. A window
     * scales once, at its edge: to pixels when drawing, back when reading
     * pointer positions. */
    gates_u32 dpi = 144; /* 150 % */
    gates_i32 px = gates_px(100, dpi);
    gates_i32 back = gates_logical(px, dpi);
    /* A theme is checked against the enforced rules: contrast and cues. */
    gates_access_issue_t issues[8];
    gates_u32 n = gates_theme_audit(gates_theme_light(), issues, 8);
    printf("100 units = %d px at %u dpi; %d px = %d units; light theme: %u issues\n", px, dpi, px, back, n);
    return n == 0 ? 0 : 1;
}
```

창은 자기 모니터(모니터별 DPI, 모니터 사이 옮기기), Windows 의 "텍스트 크기" 설정, 응용의 확대(`gates_window_set_zoom`, 25 % 부터 400 % 까지)를 따른다. 셋 모두 확대한 쪽처럼 화면 전체를 키운다.

8.2 테마

색은 값이 아니라 토큰(`gates_color_token_t`)이다. 프로그램은 "글자", "포커스", "오류"라고 말하고, 그것이 무슨 색인지는 테마가 말한다. 창은 시스템을 따른다. 밝은 모드와 어두운 모드, 시스템 자신의 색으로 만든 고대비를 따르며, 사람이 바꾸면 곧바로 바뀐다. 제목 표시줄도 따라간다. `gates_window_set_theme_mode` 는 모드를 고정하고, `gates_window_set_theme` 는 응용의 테마를 정한다. 모든 테마는 9장의 대비·표시 규칙을 지켜야 하며, `gates_theme_audit` 이 테마 하나를 검사한다.

포커스와 오류는 색으로만 보이지 않는다. 포커스 테두리와 오류 테두리에는 너비(적어도 2 단위)가 있어서 어떤 테마에서도 읽힌다.

8.3 글꼴

글은 두 글꼴 가운데 하나를 쓴다(gates/text.h 의 gates_font_t).

글꼴	무엇	Windows 에서
GATES_FONT_UI(기본)	플랫폼의 UI 글꼴, 비례폭	시스템 메시지 글꼴: Segoe UI, 한국어 Windows 에서는 맑은 고딕
GATES_FONT_MONO	고정폭. 코드, 로그, 열을 맞춰야 하는 글	Consolas
이름 붙은 글꼴(0.12.0)	gates_font_named 로 이름을 댄 글꼴	그 글꼴을 UI 크기로. 설치되어 있지 않으면 UI 글꼴

gates_node_set_font 는 노드의 글꼴을 정하고, CSS 의 font-family 처럼 그 아래에서 제 글꼴을 정하지 않은 모든 것에도 적용된다. GATES_FONT_INHERIT 는 부모의 것으로 돌아가고, gates_node_font 는 실제로 쓰이는 글꼴을 읽는다. 패널에 정하면 한 구역 전체가 바뀌고, 뷰 하나에 정하면 로그 하나가 바뀐다. 대화상자와 메뉴는 창의 루트 아래에 있지 않으므로, 따로 정하지 않으면 UI 글꼴에서 시작한다.

이름으로 글꼴을 정하는 길(0.12.0)은 둘이다. **전체를 한 글꼴로** 하려면 응용의 UI 글꼴을 정한다. 만들 때는 gates_app_desc_t.ui_font, 실행 중에는 gates_app_set_ui_font(app, name) 이다(설정 화면의 글꼴 고르기. 설치되지 않은 글꼴이면 PROVEN_ERR_NOT_FOUND 를 돌려주고, 빈 이름은 시스템 글꼴로 돌아간다). GATES_FONT_UI 를 쓰는 모든 것이, 대화상자와 메뉴까지, 그 글꼴을 시스템 UI 크기로 쓴다. **요소마다 글꼴을** 정하려면 gates_font_named(name, &face) 로 이름을 한 번 등록하고, 그 face 를 GATES_FONT_MONO 처럼 정한다. 노드 하나에, 패널에 정하면 한 구역에, 루트에 정하면 창 전체에 적용된다. 같은 이름은 늘 같은 face 를 준다. 이름은 GATES_FONT_NAMED_MAX(62)개까지다. 이름은 시스템의 글꼴 패밀리 이름이고, 영어 이름도 시스템 언어 이름도 된다(Windows 에서 "Malgun Gothic" 과 "맑은 고딕" 이 둘 다 된다).

```
gates_app_desc_t desc = { .ui_font = GATES_STR("Malgun Gothic") };  /* one face for the program */
gates_font_t serif;
if (gates_is_ok(gates_font_named(GATES_STR("Georgia"), &serif))) {
    (void)gates_node_set_font(tree, quote, serif);                  /* one element */
}
```

크기(0.10.0)는 시스템 글자 크기의 백분율이라 사람이 고른 Windows "텍스트 크기"를 그대로 따른다. gates_node_set_font_size(tree, node, percent) 는 GATES_FONT_SIZE_MIN(50)부터 GATES_FONT_SIZE_MAX(400) 사이로 크기를 정한다. 흔히 쓰는 값은 GATES_FONT_SIZE_SMALL(85), GATES_FONT_SIZE_LARGE(125), GATES_FONT_SIZE_HEADING(150)이고, 0 은 부모의 것으로 돌아간다. 크기는 글꼴처럼 물려받지만 글꼴과는 따로 다. 제목 패널 안의 MONO 뷰는 150 % 고정폭이다. 노드가 재고 그리는 모든 것이 그 크기를 따른다. 글, 줄과 행의 높이, 캐럿, 텍스트 상자의 cols 가 그렇다. gates_node_font_size 는 실제 크기를 읽고, gates_node_font 는 글꼴과 크기를 함께 돌려준다(gates_font_face, gates_font_percent).

```
(void)gates_node_set_font(tree, log_view, GATES_FONT_MONO);           /* aligned log lines */
(void)gates_node_set_font_size(tree, title, GATES_FONT_SIZE_HEADING); /* a page heading */
(void)gates_node_set_font_size(tree, note, GATES_FONT_SIZE_SMALL);    /* small print */
```

8.4 그리기와 글자 백엔드

창은 트리를 그리기 목록(gates/draw.h)에 그리고, 소프트웨어 그리기(gates/render.h)가 그것을 픽셀로 바꾼다. 글은 글자 백엔드(gates/text.h)를 거친다. 백엔드는 글꼴마다 줄 높이, 평균 글자 폭 (크기 가늠용: 텍스트 상자의 cols), 글자마다의 폭을 알린다. 글줄의 폭은 글자 폭의 합과 정확히 같고, 백엔드는 글자를 그 자리에 그린다. 그래서 어느 백엔드에서든, 어느 배율에서든 배치, 그리기, 캐럿, 누른 자리가 서로 맞는다. 내장 백엔드는 모든 글꼴이 고정폭이어서 시험이 결정적으로 돈다(크기는 8 x 16 칸을 늘이고, 이름 붙은 글꼴은 UI 글꼴로 그린다). 직접 만든 백엔드는 gates_font_face(font) 로 글꼴을, gates_font_face_name(font) 로 이름 붙은 글꼴의 이름을 읽고 gates_font_percent(font) 만큼 키운다. 응용은 창의 그리기 콜백에서 제 것을 더 그릴 수 있다. 거기서 위젯 상태를 읽지는 않는다.

9 접근성

이 장의 차례

9.1 모든 창은 눈과 포인터 없이도 읽고 쓸 수 있다 40
9.2 이름 40
9.3 강제하는 규칙 41
9.4 소리 내어 알리기 41
9.5 내레이터로 확인하기 41

헤더: `gates/access.h`.

9.1 모든 창은 눈과 포인터 없이도 읽고 쓸 수 있다

gates 는 모든 노드를 보조 기술에 설명한다. 역할(버튼, 체크박스, 편집, 목록 …), 이름, 설명, 상태 (포커스, 꺼짐, 체크, 고름, 펼침, 읽기 전용, 틀림, 필수, 화면 밖), 값, 받아들이는 동작이다. 라디오 선택지, 선택 상자의 선택지, 메뉴 항목, 뷰의 줄은 제 컨트롤의 항목이며 id 로 부른다. Windows 에서 창은 UI Automation 제공자여서 내레이터(Narrator)와 자동화 도구가 그 전부를 본다. 사람이 키보드로 할 수 있는 모든 동작을 UI Automation 으로도 할 수 있다. 누르기, 켜고 끄기, 고르기, 펼치기, 값 넣기, 스크롤, 글 읽기와 고르기가 그렇다.

보조 기술에서 온 동작은 입력과 같은 길을 간다. 같은 이벤트, 명령, 한도, 거절을 거친다. 키보드가 할 수 없는 일은 화면 읽기 프로그램도 할 수 없다.

9.2 이름

컨트롤의 접근성 이름은 이 차례로 정해진다. 직접 준 이름(`gates_node_set_access_name`), 묶인 이름표(`gates_node_set_labelled_by`, HTML 의 `<label for>` 와 같다), 폼의 이름표, 제 글(버튼의 이름, 체크박스의 글)이다. 텍스트 상자 옆에 놓은 이름표는 프로그램이 묶기 전까지는 묶이지 않는다. 감사가 그런 필드를 찾아낸다.

제 이름이 없는 메뉴는 메뉴 막대의 제목, 자기를 연 항목(하위 메뉴), 또는 프로그램이 열 때 포커스를 가진 컨트롤의 이름(맥락 메뉴)을 이름으로 쓴다(0.10.0).

```
/* manual example (host): the audit finds a field without a name; a label fixes it.
 * expect: before: 1 issue (no name); after: 0 issues, name "City" */
#include <gates/gates.h>

#include <stdio.h>

static gates_u32 audit(gates_tree_t *t, gates_access_rule_t *first) {
    (void)gates_layout_run(t, (gates_size_t){ 300, 120 }, gates_text_backend_builtin());
    gates_access_issue_t issues[8];
    gates_u32 n = gates_access_audit(t, gates_theme_light(), issues, 8);
    if (n > 0) *first = issues[0].rule;
    return n;
}

int main(void) {
    gates_tree_t *t = nullptr;
    if (!gates_is_ok(gates_tree_create(&(gates_tree_desc_t){0}, &t))) return 1;
    gates_node_t root = gates_tree_root(t), caption, city;
    if (!gates_is_ok(gates_layout_set(t, root, GATES_LAYOUT_KIND_COLUMN)) ||
        !gates_is_ok(gates_label_create(t, root, GATES_STR("City"), &caption)) ||
        !gates_is_ok(gates_textbox_create(t, root, GATES_STR(""), 20, &city))) {
```

```
        return 1;
    }
    gates_access_rule_t rule = 0;
    gates_u32 before = audit(t, &rule);
    /* A label beside a field is not tied to it until the program says so. */
    if (!gates_is_ok(gates_node_set_labelled_by(t, city, caption))) return 1;
    gates_u32 after = audit(t, &rule);
    gates_access_info_t info;
    if (!gates_is_ok(gates_access_info(t, city, 0, &info))) return 1;
    printf("before: %u issue%s (%s); after: %u issues, name \"%.*s\"\n", before, before == 1 ? "" : "s",
           rule == GATES_RULE_NO_NAME ? "no name" : "other", after, (int)info.name.size, (const char
*)info.name.ptr);
    gates_tree_destroy(t);
    return after == 0 ? 0 : 1;
}
```

폼 필드의 도움말과 오류는 설명이 된다. 그래서 화면 읽기 프로그램은 "Name, 편집, 필수, enter a name"처럼 읽는다. 자동화 id(`gates_node_set_automation_id`)는 시험 스크립트를 위한 변하지 않는 이름이며, 폼과 명령은 기본으로 `field-<id>` 와 `cmd-<id>` 를 준다.

9.3 강제하는 규칙

`gates_access_audit` 은 배치를 마친 트리를, 예제와 참조 응용이 지키는 규칙에 비추어 검사한다.

규칙	검사
NO_NAME	조작하는 컨트롤마다 접근성 이름이 있다
TARGET_SIZE	포인터 목표가 적어도 24 x 24 단위다(WCAG 2.2, 2.5.8)
KEYBOARD	조작하는 컨트롤마다 키보드로 닿을 수 있다
DUPLICATE_ID	자동화 id 가 트리 안에서 유일하다
CONTRAST	테마의 글자와 표시가 대비 비율을 지킨다
FOCUS_CUE	포커스와 오류 표시가 적어도 2 단위 넓다

감사는 시험에서 돌린다. Windows 에서는 환경의 `GATES_ACCESS_STRICT=1` 이 배치할 때마다 모든 창을 감사하고 문제 수를 제목에 보인다.

9.4 소리 내어 알리기

사람이 일하는 동안 바뀌는 상태 줄은 라이브 영역(`gates_node_set_live`)이어야 한다. 정중한(polite) 것은 지금 읽는 말 뒤에, 단호한(assertive) 것은 곧바로 읽힌다. 어떤 컨트롤에도 속하지 않는 메시지는 `gates_window_announce` 가 말한다. 숨은 시스템 캐럿이 글 캐럿을 따라가므로 돋보기는 넣는 자리를 계속 보여 준다.

9.5 내레이터로 확인하기

내레이터를 켜고(Ctrl+Windows+Enter) 키보드만으로 프로그램을 쓴다. 포커스가 닿는 컨트롤마다 역할, 이름, 상태가 읽혀야 하고, 고른 것이 옮겨 가면 목록의 줄이, 제출이 실패하면 오류가 읽혀야 한다. 내레이터의 포커스 사각형은 포커스를 가진 컨트롤 위에 있어야 한다.

10 배포와 문제 해결

이 장의 차례

10.1 링크 42
10.2 판과 호환 42
10.3 응용과 창 42
10.4 문제 해결 43

헤더: `gates/app.h`, `gates/window.h`, `gates/version.h`.

10.1 링크

대상	링크 줄
Windows 프로그램	`-lgates -lproven -lgdi32 -luser32 -limm32 -ldwmapi -ladvapi32 -luiautomationcore -lole32 -loleaut32 -luuid`, 콘솔 없는 프로그램이면 `-mwindows` 도
창 없음(어떤 시스템이든)	`-lgates_core -lproven -lm`

C23 컴파일러(`-std=c23`. GCC 14 이상, Clang 18 이상, 또는 GCC 14 이상의 mingw-w64)와 `-I<패키지>/include` 로 컴파일한다. 패키지의 `examples/consumer/` 는 패키지만으로 빌드하는 완전한 프로그램과 Makefile 이다. 원본 트리에서 `make install PREFIX=/some/dir` 은 헤더와 라이브러리를 `PREFIX/include` 와 `PREFIX/lib` 에 복사한다.

결과는 실행 파일 하나다. 라이브러리는 정적이고, gates 는 대상 기계에 Windows 10 이나 11 말고는 아무것도 요구하지 않는다(쓰는 UI Automation, 입력기, DPI 함수는 Windows 의 일부이고, 더 새 것은 실행 중에 찾는다).

10.2 판과 호환

`gates/version.h` 는 `GATES_VERSION_STRING` 과 `GATES_VERSION_NUMBER` 를 주고, `gates_version()` 은 링크한 라이브러리가 무엇인지 말한다. 마이너 판이 같은 릴리스끼리는 소스가 호환된다. 0.2.0 으로 빌드한 프로그램은 0.2.1 로도 빌드된다. 0.3.0 은 응용 프로그램 틀을 더하면서 있던 것 하나를 바꾸었다. 버튼, 체크박스, 명령의 레이블에서 영문자나 숫자 앞의 `&`는 이제 니모닉 표시이고 보이지 않는다(글자 그대로의 `&`는 `&&`로 쓴다). 0.2.0 은 글자 백엔드 계약을 바꾸었다(직접 만든 백엔드는 `glyph_advance` 를 더하고 크기 대신 글꼴을 받는다). gates 의 백엔드만 쓰는 프로그램은 다시 빌드하면 된다. 바이너리 호환은 약속하지 않는다. 공개 구조체의 크기가 바뀔 수 있으므로 판이 바뀔 때마다 프로그램을 다시 빌드하고, 시작할 때 확인한다.

```
if (gates_version() != GATES_VERSION_NUMBER) { /* headers and library differ */ }
```

10.3 응용과 창

`gates_app_create` 가 응용을 만든다(보통 프로그램에 하나, UI 스레드에서. 그 스레드를 UI Automation 을 위해 COM 단일 스레드 아파트로 초기화한다). `gates_window_create` 는 자기 트리를 가진 창을 연다. `gates_app_run` 은 마지막 창이 닫히거나 `gates_app_quit` 이 불릴 때까지 루프를 돈다. 창을 없애고 나서 응용을 없앤다. `gates_app_desc_t` 의 할당기가 gates 가 할당하는 모든 것에 쓰인다(기본 할당기는 스레드에 안전하다).

10.4 문제 해결

증상	까닭
처리기가 불리지 않는다	처리기를 달지 않았거나 프로그램이 바꾼 것이다(설정 함수는 조용하다. `gates_widget_notify` 를 쓴다)
방금 쓴 노드에 `PROVEN_ERR_INVALID_ARG` 가 온다	핸들이 낡았다. 노드가 없어졌다(그 칸이 다시 쓰였을 수도 있다)
동작에서 `PROVEN_ERR_INVALID_STATE` 가 온다	컨트롤이 꺼졌거나, 숨었거나, 모달 대화상자 뒤에 있다
입력한 글이 나타나지 않는다	텍스트 상자가 읽기 전용이거나 입력이 최대 길이를 넘었다 (LIMIT_EXCEEDED 가 답을 기다린다)
IME 가 열리지 않는다	상자가 읽기 전용이거나 비밀번호 상자다
화면이 아주 작거나 크다	크기를 논리 단위가 아니라 픽셀로 주었다(8장)
작업 스레드의 결과가 끊긴다	작업 스레드가 `GATES_POST_FULL` 을 무시했거나 `GATES_POST_CLOSED` 뒤에도 보냈다
내레이터가 이름 없이 "편집"이라고 읽는다	필드 옆의 이름표가 묶이지 않았다(9장)
링크할 때 `proven_*` 기호가 겹친다	프로그램이 proven 사본 둘을 링크한다. 하나만 남긴다

11 응용 프로그램 틀

이 장의 차례

11.1 니모닉 44
11.2 메뉴 막대 44
11.3 키맵: 단축키 나열과 다시 묶기 45
11.4 도구 막대 46
11.5 상태 줄 46
11.6 툴팁 47
11.7 탭 48
11.8 배치 기억하기 48
11.9 접근성 50

헤더: `gates/frame.h`, `gates/command.h`, `gates/state.h`.

데스크톱 도구는 보통 내용 둘레에 틀을 갖는다. 메뉴 막대가 있고, 레이블과 버튼에는 키보드 접근 키가 있으며, 사람이 익히고 바꿀 수 있는 단축키가 있다. 이 모두는 4장의 명령으로 움직이므로, 한 동작은 어디에 나타나든 레이블 하나, 사용 가능 상태 하나, 단축키 하나를 지킨다.

11.1 니모닉

버튼, 체크 상자, 명령, 메뉴 막대 제목, 탭의 글자에서 `&x`는 `x`가 영문자나 숫자일 때 `x`를 니모닉으로 표시한다. 예를 들면 `"&Save"`, `"E&xit"`, `"Page &2"`이다. 그러면 Alt+x가 그 컨트롤을 작동시킨다. 버튼은 눌리고, 체크 상자는 바뀌고, 메뉴 막대 제목은 열린다. `&` 두 개는 하나로 보이고(`"Fish && chips"`), 그 밖의 `&`는 그대로 보이므로 `"Save & close"`는 따로 처리할 필요가 없다. 니모닉 표시는 자리를 차지하지 않는다. `"&Save"`는 `"Save"`와 너비가 똑같다.

그 글자는 다른 Windows 프로그램처럼 키보드 표시가 보이는 동안 밑줄이 그어진다. Alt를 누르거나 키보드로 메뉴 막대에 들어간 때부터 다음 포인터 누름까지이고, "선택키 밑줄 표시" 설정이 켜져 있으면 늘 그어진다.

레이블은 파일 이름 같은 자료를 보여 주는 일이 많으므로, 대상이 있을 때만 `&`를 해석한다. `gates_label_set_target(tree, label, box)`를 부르면 Alt+x가 `box`에 초점을 준다. 그 상자에 레이블을 접근성 이름으로도 주면(`gates_node_set_labelled_by`) 화면 낭독기가 이름과 함께 키를 읽는다.

여러 컨트롤이 같은 글자를 쓰면 Alt+x를 누를 때마다 초점이 다음 컨트롤로 옮겨 가고 아무것도 작동하지 않는다. 사람이 Space나 Enter를 누른다. 키보드가 닿는 컨트롤만 센다. 쓸 수 없는 버튼, 숨은 패널, 모달 대화상자 뒤의 창은 건너뛴다. 니모닉은 ASCII 영문자와 숫자이다. 다른 문자에는 `"파일(&F)"`처럼 괄호 안에 하나 넣는다.

11.2 메뉴 막대

트리에는 메뉴 막대가 많아야 하나 있다. `gates_menubar_create(tree, parent, scope, &bar)`로 만든다. `gates_menubar_add(tree, bar, title, ids, count, nullptr)`를 부를 때마다 `scope`의 명령 목록 위에 제목 하나가 더해지고, id 0은 구분선을 그린다. 문맥 메뉴가 받는 목록과 같다. 메뉴는 보통 메뉴 오버레이이므로 4장에서 메뉴에 대해 말한 것이 모두 성립한다. Escape, 바깥 클릭, 창이 초점을 잃을 때 닫히고, 고른 항목은 명령을 실행하기 전에 다시 확인되며, 열린 메뉴에 처리기를 두면 MENU_CLOSED를 받는다.

포인터로는 제목을 클릭하면 그 아래에 메뉴가 열린다. 하나가 열려 있는 동안 다른 제목 위로 움직이면 그 메뉴로 바뀌고, 열린 제목을 클릭하면 닫힌다. 키보드로는 다음과 같다.

키	메뉴 모드(제목 하나가 강조됨)	열린 메뉴
F10, 또는 Alt만 눌렀다 뗌	메뉴 모드에 들어감 / 나감	나감
Left / Right	앞 / 다음 제목	이웃 메뉴를 엶
Down, Up, Enter, Space	강조된 제목을 엶	이동 / 고름
제목의 니모닉 글자	그 제목을 엶	-
항목의 니모닉 글자	-	그 항목을 고름
Escape	메뉴 모드에서 나감	강조된 제목으로 돌아감

제목의 글자로 Alt+x를 누르면 그 메뉴가 곧바로 열린다. 제목이 컨트롤보다 먼저이므로 컨트롤에는 제목이 쓰지 않는 글자를 준다. F10은 메뉴 막대가 닿을 수 있는 동안만 메뉴 키이다. 메뉴 막대가 없는 창에서는 보통 명령 단축키로 남는다. 막대는 Tab 정지 위치가 아니며, 모달 대화 상자가 있으면 닿을 수 없다. 이 판에서 메뉴에는 하위 메뉴가 없다.

Windows에서 gates는 Alt+글자가 니모닉과 맞을 때만 받는다. Alt+F4, Alt+Tab, Alt+Space는 Windows가 하는 일을 그대로 한다.

11.3 키맵: 단축키 나열과 다시 묶기

각 명령에 저장된 단축키가 곧 키맵이다. `gates_command_count`와 `gates_command_at`은 한 범위의 명령을 나열한다(키맵 편집기나 도움말 화면에 쓴다). `gates_command_set_shortcut`은 하나를 다시 묶고, 그 범위의 다른 명령이 이미 가진 단축키는 거절하며 그 명령을 `*conflict`에 알려 준다. `gates_shortcut_format`은 단축키를 메뉴와 화면 낭독기가 보여 주는 대로 적고(`"Ctrl+Shift+S"`, `"F5"`, `"Ctrl+Del"`), `gates_shortcut_parse`는 같은 글자를 다시 읽는다. 그래서 프로그램은 사람의 키맵을 `save=Ctrl+S` 같은 줄로 된 파일에 둘 수 있다. 4장의 규칙은 그대로이다. 단축키에는 Ctrl이 있어야 하거나 기능 키여야 하고, Alt는 단축키에 들어가지 않는다.

```
/* manual example (host): a menu bar, its keyboard and its mnemonics.
 * expect: new 1, open 1; F10 highlights File; access key Alt+F; shortcut Ctrl+Shift+N */
#include <gates/gates.h>

#include <stdio.h>

enum { CMD_NEW = 1, CMD_OPEN, CMD_QUIT };

static void on_command(gates_tree_t *tree, gates_command_id_t id, void *user) {
    (void)tree;
    ((int *)user)[id]++;
}

static bool press(gates_tree_t *t, gates_key_t key) {
    gates_key_event_t e = { .key = key, .down = true };
    return gates_input_key(t, &e);
}

int main(void) {
    gates_tree_t *t = nullptr;
    if (!gates_is_ok(gates_tree_create(&(gates_tree_desc_t){0}, &t))) return 1;
    gates_node_t root = gates_tree_root(t), bar;
    int runs[4] = {0};
    /* "&" marks the mnemonic: Alt+N, and N while the menu is open. */
    gates_command_desc_t cmds[] = {
        { .id = CMD_NEW, .label = GATES_STR("&New"), .shortcut = { .letter = 'N', .ctrl = true },
          .enabled = true, .invoke = on_command, .user = runs },
        { .id = CMD_OPEN, .label = GATES_STR("&Open..."), .enabled = true, .invoke = on_command, .user = runs },
        { .id = CMD_QUIT, .label = GATES_STR("E&xit"), .enabled = true, .invoke = on_command, .user = runs },
    };
    for (int i = 0; i < 3; i++) {
        if (!gates_is_ok(gates_command_register(t, root, &cmds[i]))) return 1;
```

```
    }
    static const gates_command_id_t file[] = { CMD_NEW, CMD_OPEN, 0, CMD_QUIT }; /* 0: separator */
    if (!gates_is_ok(gates_layout_set(t, root, GATES_LAYOUT_KIND_COLUMN)) ||
        !gates_is_ok(gates_menubar_create(t, root, root, &bar)) ||
        !gates_is_ok(gates_menubar_add(t, bar, GATES_STR("&File"), file, 4, nullptr)) ||
        !gates_is_ok(gates_layout_run(t, (gates_size_t){ 400, 300 }, gates_text_backend_builtin()))) {
        return 1;
    }
    /* F10 enters menu mode, Down opens File on its first entry, Enter chooses it. */
    (void)press(t, GATES_KEY_F10);
    bool highlighted = gates_menubar_highlighted(t, bar) == 0;
    (void)press(t, GATES_KEY_DOWN);
    (void)press(t, GATES_KEY_ENTER);
    /* Alt+F opens File directly (the platform sends Alt+letter here); O chooses Open. */
    (void)gates_input_mnemonic(t, 'f');
    gates_key_event_t o = { .letter = 'O', .down = true };
    (void)gates_input_key(t, &o);
    (void)gates_tree_dispatch_events(t, 0);

    /* The keymap: rebind New, and print the shortcut as menus do. */
    gates_command_id_t conflict = 0;
    if (!gates_is_ok(gates_command_set_shortcut(t, root, CMD_NEW,
                                                (gates_shortcut_t){ .letter = 'N', .ctrl = true, .shift = true },
                                                &conflict))) {
        return 1;
    }
    char keys[32];
    gates_shortcut_t now = gates_command_shortcut(t, root, CMD_NEW);
    (void)gates_shortcut_format(&now, keys, sizeof keys);

    gates_access_info_t info;
    if (!gates_is_ok(gates_access_info(t, bar, 1, &info))) return 1;   /* item 1: the File title */
    printf("new %d, open %d; F10 %s File; access key %.*s; shortcut %s\n", runs[CMD_NEW], runs[CMD_OPEN],
           highlighted ? "highlights" : "misses", (int)info.access_key.size, (const char *)info.access_key.ptr,
           keys);
    gates_tree_destroy(t);
    return 0;
}
```

11.4 도구 막대

`gates_toolbar_create(tree, parent, scope, &bar)`는 작은 버튼이 늘어선 줄을 만들고, `gates_toolbar_add(tree, bar, id)`를 부를 때마다 `scope`의 명령에 묶인 버튼이 하나씩 더해진다(id 0은 구분선). 버튼은 명령의 레이블을 니모닉 표시 없이 보여 주고, 명령을 쓸 수 없는 동안은 흐리게, 명령이 체크된 동안은 눌린 모습으로 보인다. 굵게 같은 켜고 끄는 버튼은 체크된 명령이다(0.14.0 부터 강조색 테두리). 버튼은 포인터가 위에 올 때까지 평평하고, 아래에 선이 그어진 띠 위에 놓이므로 글자 버튼 줄이 탭 줄처럼 읽히지 않는다. 버튼마다 아이콘(`gates_command_set_icon`)을 달면 더 뚜렷이 구별된다.

클릭은 명령을 실행하고 키보드 초점은 그대로 둔다. 그래서 잘라내기와 붙여넣기 버튼은 사람이 입력하던 글상자에 작용한다. 키보드로는 도구 막대 전체가 Tab 정지 위치 하나이다. Left, Right, Home, End가 쓸 수 있는 버튼 사이를 옮기고, Space나 Enter가 초점이 있는 버튼을 실행한다. 도구 막대가 버튼보다 좁으면 들어가지 않는 버튼은 빠지고, 끝의 >> 버튼이 그 명령들을 메뉴로 보여 준다.

11.5 상태 줄

`gates_statusbar_create(tree, parent, &bar)`는 창 아래쪽을 따라 놓이는 띠를 만들고, `gates_statusbar_add(tree, bar, text, grow, &segment)`는 구역 하나를 더한다. 구역은 레이블이므로 글자는 `gates_widget_set_text`로 바꾼다. 구역 사이에는 가는 선이 있고, `grow`가 있는 구역은 레이아웃 자식처럼 남는 너비를 나눠 가진다. 상태 글자는 바뀔 때마다 읽히지 않는다. 소식이 중요한 구역만 라이브 영역으로 만든다(`gates_node_set_live`).

11.6 툴팁

`gates_node_set_tooltip(tree, node, text)`는 노드에 짧은 도움말을 준다. 포인터가 `GATES_TOOLTIP_DELAY_MS`(500 ms) 동안 머물거나 키보드 초점이 온 뒤 그만큼 지나면 노드 아래(창 아래쪽 가까이에서는 위)의 작은 상자에 나타나고, 누름, 키, 포인터나 초점이 떠날 때, 그리고 `GATES_TOOLTIP_SHOW_MS`가 지나면 숨는다. 툴팁 하나가 보이는 동안 다른 툴팁으로 옮기면 곧바로 바뀐다. 도구 막대 버튼은 따로 부탁하지 않아도 명령의 레이블과 단축키로 된 툴팁을 가진다(`"Paste (Ctrl+V)"`). 툴팁은 입력을 받지 않으며, 화면 낭독기는 그 글자를 노드의 도움말로 읽고, 보이는 툴팁은 열릴 때 알려지는 ToolTip 요소이기도 하다. 모달 대화상자가 열리면 툴팁은 숨는다. 툴팁은 트리의 시계로 시간을 재며, 포인터가 올라가 있거나 초점이 있는 노드에 툴팁이 없는 동안에는 아무것도 재지 않는다.

```
/* manual example (host): a toolbar, a status bar and a tooltip.
 * expect: pasted 1, focus kept; tooltip "Paste (Ctrl+V)" after 500 ms; status Pasted */
#include <gates/gates.h>

#include <stdio.h>
#include <string.h>

enum { CMD_CUT = 1, CMD_PASTE };

typedef struct app_t {
    gates_node_t status;
    int pasted;
} app_t;

static void on_command(gates_tree_t *tree, gates_command_id_t id, void *user) {
    app_t *a = user;
    if (id == CMD_PASTE) {
        a->pasted++;
        (void)gates_widget_set_text(tree, a->status, GATES_STR("Pasted"));
    }
}

/* A test clock: a window's tree has a real one. */
static gates_u64 now_ms;
static gates_u64 clock_now(void *ctx) { (void)ctx; return now_ms; }
static void clock_changed(void *ctx) { (void)ctx; }

int main(void) {
    gates_tree_t *t = nullptr;
    if (!gates_is_ok(gates_tree_create(&(gates_tree_desc_t){0}, &t))) return 1;
    gates_tree_set_clock(t, clock_now, clock_changed, nullptr);
    gates_node_t root = gates_tree_root(t), bar, body, status_bar;
    app_t a = {0};
    gates_command_desc_t cmds[] = {
        { .id = CMD_CUT, .label = GATES_STR("Cu&t"), .shortcut = { .key = GATES_KEY_X, .ctrl = true },
          .enabled = true, .invoke = on_command, .user = &a },
        { .id = CMD_PASTE, .label = GATES_STR("&Paste"), .shortcut = { .key = GATES_KEY_V, .ctrl = true },
          .enabled = true, .invoke = on_command, .user = &a },
    };
    if (!gates_is_ok(gates_layout_set(t, root, GATES_LAYOUT_KIND_COLUMN)) ||
        !gates_is_ok(gates_command_register(t, root, &cmds[0])) ||
        !gates_is_ok(gates_command_register(t, root, &cmds[1])) ||
        !gates_is_ok(gates_toolbar_create(t, root, root, &bar)) ||
        !gates_is_ok(gates_toolbar_add(t, bar, CMD_CUT)) ||
        !gates_is_ok(gates_toolbar_add(t, bar, CMD_PASTE)) ||
        !gates_is_ok(gates_textbox_create(t, root, GATES_STR(""), 30, &body)) ||
        !gates_is_ok(gates_node_set_access_name(t, body, GATES_STR("Text"))) ||
        !gates_is_ok(gates_layout_set_child_grow(t, body, 1)) ||
        !gates_is_ok(gates_statusbar_create(t, root, &status_bar)) ||
```

```
        !gates_is_ok(gates_statusbar_add(t, status_bar, GATES_STR("Ready"), 1, &a.status)) ||
        !gates_is_ok(gates_layout_run(t, (gates_size_t){ 400, 300 }, gates_text_backend_builtin()))) {
        return 1;
    }
    gates_tree_set_focus(t, body);

    /* Find the Paste button through the accessibility model: item 2 of the bar. */
    gates_access_info_t info;
    if (!gates_is_ok(gates_access_info(t, bar, 2, &info))) return 1;
    gates_point_t p = { info.bounds.x + info.bounds.w / 2, info.bounds.y + info.bounds.h / 2 };

    /* Rest the pointer on it: the tooltip shows after GATES_TOOLTIP_DELAY_MS. */
    gates_pointer_event_t move = { .action = GATES_POINTER_MOVE, .pos = p };
    (void)gates_input_pointer(t, &move);
    now_ms += GATES_TOOLTIP_DELAY_MS;
    (void)gates_tree_run_timers(t);
    gates_str_t tip = {0};
    char tip_text[64] = "";
    if (gates_tooltip_shown(t, nullptr, nullptr, &tip, nullptr) && tip.size < sizeof tip_text) {
        memcpy(tip_text, tip.ptr, tip.size);
    }

    /* A click invokes the command; the focus stays in the text box. */
    gates_pointer_event_t down = { .action = GATES_POINTER_DOWN, .button = GATES_BUTTON_LEFT, .pos = p };
    gates_pointer_event_t up = { .action = GATES_POINTER_UP, .button = GATES_BUTTON_LEFT, .pos = p };
    (void)gates_input_pointer(t, &down);
    (void)gates_input_pointer(t, &up);
    (void)gates_tree_dispatch_events(t, 0);

    gates_str_t s = gates_widget_text(t, a.status);
    printf("pasted %d, focus %s; tooltip \"%s\" after %u ms; status %.*s\n", a.pasted,
           gates_node_eq(gates_tree_focus(t), body) ? "kept" : "moved", tip_text, GATES_TOOLTIP_DELAY_MS,
           (int)s.size, (const char *)s.ptr);
    gates_tree_destroy(t);
    return 0;
}
```

11.7 탭

`gates_tabs_create(tree, parent, &tabs)`는 페이지 위에 제목이 늘어선 띠를 만들고, `gates_tabs_add(tree, tabs, title, &page)`를 부를 때마다 탭 하나가 더해지며 그 페이지를 돌려준다. 페이지는 컨트롤을 담는 세로 패널이다. 한 번에 한 페이지만 보이고, 다른 페이지의 컨트롤은 포인터, 키보드, 보조 기술이 닿지 않는다. 고른 탭과 그 페이지는 한 장이다(0.14.0). 페이지에 테두리가 둘리고, 고른 탭은 위쪽에 강조색 막대를 달고 페이지로 열리며, 다른 탭은 그 뒤의 윤곽선이다.

띠 전체가 Tab 정지 위치 하나이다. Left, Right, Home, End는 곧바로 고르고, Ctrl+Tab과 Ctrl+Shift+Tab(또는 Ctrl+PgDn과 Ctrl+PgUp)은 탭 안 어디서나 바꾸며 끝에서 처음으로 돌아간다. 클릭은 고르고 띠에 초점을 주며, 제목의 니모닉은 그 탭을 고른다. 초점이 사라지는 페이지에 있었으면 새 페이지의 첫 컨트롤로, 없으면 띠로 옮겨 간다. 사람이 바꾸면 탭 노드에 GATES_EVENT_VALUE_CHANGED가 오고 `result`는 새 번호이다. `gates_tabs_set_selected`는 조용히 바꾼다. 제목이 다 들어가지 않으면 띠는 온전한 제목만, 늘 고른 탭을 포함해 보이고 >> 버튼을 둔다. 그 버튼을 누르거나 띠에서 Alt+Down 을 누르면 고른 탭에 표시가 된 모든 탭의 메뉴가 열린다.

11.8 배치 기억하기

분할선을 끌고, 탭을 고르고, 열을 넓히고, 스크롤한 사람은 다음에도 그대로이기를 바란다. `gates_state_save`는 자동화 id가 있는 모든 노드의 그런 배치를 글자로 적고(`split`, `tabs`, `columns`, `scroll` 줄), `gates_state_load`는 그 글자를 새로 만든 트리에 적용한다. 글자를 어디에 둘지는 프로그램이 정하며, 보통 설정 옆의 파일이다.

id가 열쇠이므로 판이 바뀌어도 id를 그대로 둔다. 더는 맞지 않는 줄은 건너뛰므로 오래된 파일이 새 프로그램을 망가뜨리지 않는다. Windows에서는 `gates_window_placement`와 `gates_window_set_placement`가 창의 위치, 크기, 최대화 상태를 같은 방식으로 다루고, 모니터가 사라진 창은 가장 가까운 모니터로 데려온다.

```
/* manual example (host): tabs, and state kept between runs.
 * expect: tab 1 (Advanced) after Ctrl+Tab; saved 2 lines; next run: tab 1, split 700 */
#include <gates/gates.h>

#include <stdio.h>

/* Builds the same window each run: tabs and a split, both with automation ids. */
static bool build(gates_tree_t **out, gates_node_t *tabs, gates_node_t *split, gates_node_t *apply) {
    gates_tree_t *t = nullptr;
    if (!gates_is_ok(gates_tree_create(&(gates_tree_desc_t){0}, &t))) return false;
    gates_node_t root = gates_tree_root(t), page, left, right, reset;
    bool ok = gates_is_ok(gates_layout_set(t, root, GATES_LAYOUT_KIND_COLUMN)) &&
              gates_is_ok(gates_tabs_create(t, root, tabs)) &&
              gates_is_ok(gates_tabs_add(t, *tabs, GATES_STR("&General"), &page)) &&
              gates_is_ok(gates_button_create(t, page, GATES_STR("Apply"), nullptr, nullptr, apply)) &&
              gates_is_ok(gates_tabs_add(t, *tabs, GATES_STR("&Advanced"), &page)) &&
              gates_is_ok(gates_button_create(t, page, GATES_STR("Reset"), nullptr, nullptr, &reset)) &&
              gates_is_ok(gates_node_set_automation_id(t, *tabs, GATES_STR("settings.tabs"))) &&
              gates_is_ok(gates_panel_create(t, root, split)) &&
              gates_is_ok(gates_layout_set(t, *split, GATES_LAYOUT_KIND_SPLIT)) &&
              gates_is_ok(gates_panel_create(t, *split, &left)) &&
              gates_is_ok(gates_panel_create(t, *split, &right)) &&
              gates_is_ok(gates_node_set_automation_id(t, *split, GATES_STR("main.split"))) &&
              gates_is_ok(gates_layout_run(t, (gates_size_t){ 400, 300 }, gates_text_backend_builtin()));
    *out = t;
    return ok;
}

int main(void) {
    gates_tree_t *t;
    gates_node_t tabs, split, apply;
    if (!build(&t, &tabs, &split, &apply)) return 1;
    /* The person works in General, then Ctrl+Tab switches to Advanced. */
    gates_tree_set_focus(t, apply);
    gates_key_event_t ctrl_tab = { .key = GATES_KEY_TAB, .ctrl = true, .down = true };
    (void)gates_input_key(t, &ctrl_tab);
    gates_u32 now = gates_tabs_selected(t, tabs);
    gates_str_t title = gates_tabs_title(t, tabs, now);
    /* ...and drags the split. Save the state (a real program writes it to a file). */
    if (!gates_is_ok(gates_layout_set_split(t, split, GATES_SPLIT_HORIZONTAL, 700))) return 1;
    gates_u8 text[256];
    gates_usize_t len = 0;
    if (!gates_is_ok(gates_state_save(t, text, sizeof text, &len))) return 1;
    int lines = 0;
    for (gates_usize_t i = 0; i < len; i++) lines += text[i] == '\n' && i > 16;
    gates_tree_destroy(t);

    /* The next run builds the window again and loads what was saved. */
    if (!build(&t, &tabs, &split, &apply)) return 1;
    if (!gates_is_ok(gates_state_load(t, (gates_str_t){ .ptr = text, .size = len }, nullptr))) return 1;
    printf("tab %u (%.*s) after Ctrl+Tab; saved %d lines; next run: tab %u, split %d\n", now,
           (int)title.size - 1, (const char *)title.ptr + 1, lines, gates_tabs_selected(t, tabs),
           gates_layout_split_ratio(t, split));
    gates_tree_destroy(t);
    return 0;
}
```

11.9 접근성

메뉴 막대는 MenuBar 요소이고 그 제목은 ExpandCollapse가 있는 메뉴 항목이다. 메뉴 모드에서는 강조된 제목이 키보드 초점을 가진다. 니모닉이 있는 컨트롤은 모두 그것을 접근 키로 알린다("Alt+F", 열린 메뉴의 항목은 글자만). 메뉴 항목과 명령에 묶인 버튼은 명령의 단축키를 가속 키로 알리며, 접근성 이름에는 `&` 표시가 들어가지 않는다. 도구 막대는 버튼을 항목으로 가진 ToolBar 요소이고(구분선은 항목이 아니며, `>>`는 보이는 동안 More라는 항목이다), 상태 줄은 구역을 글자로 가진 StatusBar 요소이다. 탭 띠는 제목을 SelectionItem이 있는 TabItem 항목으로 가진 Tab 요소이고, 각 페이지는 탭 이름을 가진 그룹이다.

12 숫자, 묶음, 배치

이 장의 차례

12.1 여러 컨트롤에 처리기 하나 51
12.2 한 차례에 한 번: 미룬 호출 51
12.3 스핀 상자와 슬라이더 51
12.4 그룹 상자 53
12.5 격자와 줄바꿈 배치 53
12.6 접근성 54

헤더: `gates/inputs.h`, `gates/event.h`, `gates/widget.h`, `gates/layout.h`.

12.1 여러 컨트롤에 처리기 하나

계산기에는 버튼이 스무 개 있고, 설정 페이지에는 모두 "무언가 바뀌었다"를 뜻하는 필드가 열몇 개 있다. `gates_node_set_bubble_handler(tree, node, fn, user)`는 패널, 폼, 루트 같은 담는 노드에 처리기 하나를 두고, 그 처리기는 자기 처리기가 없는 모든 자손의 이벤트를 듣는다. `ev->source`가 어느 컨트롤인지 알려 준다. 가장 가까운 담는 노드만 이벤트를 들으며, 오버레이는 따로 된 루트이다. 대화 상자의 버튼은 창의 처리기에 닿지 않고 대화 상자 안의 처리기에만 닿는다.

12.2 한 차례에 한 번: 미룬 호출

어떤 일은 변경 하나하나가 아니라 변경이 몰린 뒤에 해야 한다. 여러 필드의 합계, 목록의 거르기가 그렇다. `gates_tree_defer(tree, key, fn, user)`는 다음 안전한 때, 곧 지금 쌓인 이벤트를 전한 뒤에 `fn`이 한 번 돌게 하며, 그 전에 몇 번을 부탁해도 한 번이다. key가 일을 가리키고 `gates_tree_cancel_defer`가 그 일을 버린다. 미룬 일이 도는 동안 부탁한 일은 다음 차례를 기다리므로 일 하나가 프로그램을 계속 붙잡을 수 없다.

12.3 스핀 상자와 슬라이더

둘 다 범위 안의 정수를 고친다(`gates_range_t`: `min`, `max`, `step`, `page`, `value`). 소수 양은 `scale`을 쓴다. scale 100이면 값 125가 `1.25`로 보이므로 돈과 치수가 부동 소수점 반올림을 만나지 않는다. 사람이 바꾸면 `ev->value`를 담은 GATES_EVENT_VALUE_CHANGED가 오고, `gates_range_set_value`는 값을 조용히 바꾸며 `gates_range_set`은 한계를 바꾼다.

`gates_spin_create`는 위아래 화살표가 달린 글상자를 만든다. Up과 Down이 한 칸, PgUp과 PgDn이 한 쪽씩 움직이고, 화살표는 클릭으로 한 칸 움직이고 누르고 있으면 되풀이한다. Left, Right, Home, End는 글자에 남는다. 휠은 스핀 상자나 슬라이더에 포커스가 있을 때만 값을 바꾼다. 포커스가 없으면 쪽을 스크롤하므로, 폼을 읽다가 값이 뜻하지 않게 바뀌는 일은 없다. 입력한 글자가 범위 안의 숫자가 아닌 동안 상자는 잘못됨으로 표시되고, Enter나 상자를 떠나는 것이 그 글자를 확정한다. 그때도 범위 안의 숫자가 아니면 값으로 되돌아간다. `gates_spin_box`는 글상자를 돌려준다(레이블의 대상이나 너비에 쓴다).

`gates_slider_create`는 손잡이가 있는 트랙을 가로나 세로로 만든다(세로는 위가 더 크다). 화살표가 한 칸, PgUp과 PgDn이 한 쪽, Home과 End가 끝으로 가고, 손잡이를 끌면 온 칸 단위로 움직이며, 트랙을 누르면 포인터 쪽으로 한 쪽 간다. `gates_slider_set_ticks`는 n칸마다 눈금을 그린다.

`gates_range_format`과 `gates_range_parse`는 스핀 상자와 같은 방식으로 값과 글자를 바꾼다(,는 .로 읽는다).

```
/* manual example (host): a spin box and a slider, one bubble handler, one deferred total.
 * expect: width 2.50, volume 30; the panel heard 2 reports, the total was computed 1 time: 2.50 x 30 = 75.00 */
#include <gates/gates.h>
```

```
#include <stdio.h>

typedef struct app_t {
    gates_node_t width, volume;
    int heard, totals;
    char total[128];
} app_t;

/* Runs once after all the changes of a turn, however many there were. */
static void recompute(gates_tree_t *tree, gates_u32 key, void *user) {
    (void)key;
    app_t *a = user;
    a->totals++;
    char w[32];
    gates_i64 wv = gates_range_value(tree, a->width), vv = gates_range_value(tree, a->volume);
    (void)gates_range_format(wv, 100, w, sizeof w);
    char t[32];
    (void)gates_range_format(wv * vv, 100, t, sizeof t);
    snprintf(a->total, sizeof a->total, "%s x %lld = %s", w, (long long)vv, t);
}

/* One handler on the panel hears every control in it (ev->source says which). */
static void on_change(gates_tree_t *tree, const gates_event_t *ev, void *user) {
    app_t *a = user;
    if (ev->kind != GATES_EVENT_VALUE_CHANGED) return;
    a->heard++;
    (void)gates_tree_defer(tree, 1, recompute, a);
}

static bool key(gates_tree_t *t, gates_key_t k) {
    gates_key_event_t e = { .key = k, .down = true };
    return gates_input_key(t, &e);
}

int main(void) {
    gates_tree_t *t = nullptr;
    if (!gates_is_ok(gates_tree_create(&(gates_tree_desc_t){0}, &t))) return 1;
    gates_node_t root = gates_tree_root(t);
    app_t a = {0};
    /* Width in hundredths: 1.50 to 5.00 in steps of 0.25. */
    gates_range_t width = { .min = 150, .max = 500, .step = 25, .value = 200, .scale = 100 };
    gates_range_t volume = { .min = 0, .max = 100, .step = 5, .page = 20, .value = 10 };
    if (!gates_is_ok(gates_layout_set(t, root, GATES_LAYOUT_KIND_COLUMN)) ||
        !gates_is_ok(gates_spin_create(t, root, &width, &a.width)) ||
        !gates_is_ok(gates_slider_create(t, root, &volume, false, &a.volume)) ||
        !gates_is_ok(gates_node_set_bubble_handler(t, root, on_change, &a)) ||
        !gates_is_ok(gates_layout_run(t, (gates_size_t){ 400, 200 }, gates_text_backend_builtin()))) {
        return 1;
    }
    /* The person: two steps up in the spin box, a page up on the slider. */
    gates_tree_set_focus(t, gates_spin_box(t, a.width));
    (void)key(t, GATES_KEY_UP);
    (void)key(t, GATES_KEY_UP);
    gates_tree_set_focus(t, a.volume);
    (void)key(t, GATES_KEY_PAGE_UP);
    (void)gates_tree_dispatch_events(t, 0);

    char w[32];
    (void)gates_range_format(gates_range_value(t, a.width), 100, w, sizeof w);
    /* The spin box's two steps coalesce into one report of the latest value. */
    printf("width %s, volume %lld; the panel heard %d reports, the total was computed %d time: %s\n", w,
           (long long)gates_range_value(t, a.volume), a.heard, a.totals, a.total);
```

```
    gates_tree_destroy(t);
    return 0;
}
```

12.4 그룹 상자

gates_group_create(tree, parent, title, collapsible, &group, &content)는 세로 패널 둘레에 제목이 있는 틀을 그린다. 관련된 컨트롤을 content에 넣는다. 제목은 니모닉 표시를 받는다. 보통 그룹에서 Alt+x는 대상이 있는 레이블처럼 안의 첫 컨트롤로 간다. 접을 수 있는 그룹의 제목은 열림/닫힘 표시가 있는 Tab 정지 위치이다. Space, Enter, 클릭, 니모닉이 내용을 보이거나 숨기며, 숨은 내용의 컨트롤은 자리를 차지하지 않고 닿을 수도 없다. 사람이 바꾸면 그룹에 VALUE_CHANGED가 오고 ev->checked는 펼쳐졌는지이다. gates_group_set_expanded는 조용히 바꾼다.

12.5 격자와 줄바꿈 배치

GATES_LAYOUT_KIND_GRID는 자식을 gates_layout_set_grid 개의 열로 된 행에 놓는다(기본 두 열). 열은 가장 넓은 자식만큼 넓고, 행은 가장 높은 자식만큼 높으며, 자식은 행 안에서 세로 가운데에 놓이고 정렬이 자기 너비를 청하지 않으면 칸의 너비를 채운다. gates_layout_set_grid_column_grow는 남는 너비를 열에 나눠 준다. 흔히 레이블은 고정 열에, 필드는 늘어나는 열에 둔다. gates_layout_set_grid_row_grow는 남는 높이를 행(처음 GATES_GRID_MAX_GROW_ROWS 개)에 나눠 준다. gates_layout_set_child_span은 자식이 여러 열을 덮게 한다. GATES_LAYOUT_KIND_WRAP는 자식을 제 크기로 왼쪽에서 오른쪽으로 놓고, 다음 자식이 들어가지 않으면 종이 위의 낱말처럼 새 줄을 시작한다. 그 높이는 받은 너비를 따른다. 두 배치 모두 간격은 두 방향에 쓰인다.

```
/* manual example (host): a grid form inside a collapsible group, and wrapped chips.
 * expect: labels in column 1, fields in column 2 at x 40; 7 chips on 3 lines; folded: 0 fields reachable */
#include <gates/gates.h>

#include <stdio.h>

int main(void) {
    gates_tree_t *t = nullptr;
    if (!gates_is_ok(gates_tree_create(&(gates_tree_desc_t){0}, &t))) return 1;
    gates_node_t root = gates_tree_root(t), group, content, grid, chips, field[2], chip[7];
    const gates_text_backend_t *be = gates_text_backend_builtin(); /* 8 units a character */
    bool ok = gates_is_ok(gates_layout_set(t, root, GATES_LAYOUT_KIND_COLUMN)) &&
              gates_is_ok(gates_group_create(t, root, GATES_STR("&Connection"), true, &group, &content)) &&
              gates_is_ok(gates_panel_create(t, content, &grid)) &&
              gates_is_ok(gates_layout_set(t, grid, GATES_LAYOUT_KIND_GRID)) &&   /* two columns */
              gates_is_ok(gates_layout_set_gap(t, grid, 8)) &&
              gates_is_ok(gates_layout_set_grid_column_grow(t, grid, 1, 1));    /* fields take the rest */
    static const char *names[2] = { "Host", "Port" };
    for (int i = 0; ok && i < 2; i++) {
        gates_node_t l;
        ok = gates_is_ok(gates_label_create(t, grid, (gates_str_t){ .ptr = (const gates_u8 *)names[i], .size =
4 }, &l)) &&
             gates_is_ok(gates_textbox_create(t, grid, GATES_STR(""), 12, &field[i])) &&
             gates_is_ok(gates_node_set_labelled_by(t, field[i], l));
    }
    ok = ok && gates_is_ok(gates_panel_create(t, root, &chips)) &&
         gates_is_ok(gates_layout_set(t, chips, GATES_LAYOUT_KIND_WRAP)) &&
         gates_is_ok(gates_layout_set_gap(t, chips, 4));
    for (int i = 0; ok && i < 7; i++) {
        ok = gates_is_ok(gates_button_create(t, chips, GATES_STR("tag"), nullptr, nullptr, &chip[i]));
    }
    if (!ok || !gates_is_ok(gates_layout_run(t, (gates_size_t){ 160, 400 }, be))) return 1;

    gates_rect_t g = gates_node_layout_rect(t, grid), f = gates_node_layout_rect(t, field[0]);
    int lines = 1;
```

```
    for (int i = 1; i < 7; i++) {
        lines += gates_node_layout_rect(t, chip[i]).y > gates_node_layout_rect(t, chip[i - 1]).y;
    }
    /* Folding the group hides its content: nothing inside takes room or focus. */
    if (!gates_is_ok(gates_group_set_expanded(t, group, false)) ||
        !gates_is_ok(gates_layout_run(t, (gates_size_t){ 160, 400 }, be))) {
        return 1;
    }
    int shown = gates_widget_focusable(t, field[0]) + gates_widget_focusable(t, field[1]);
    printf("labels in column 1, fields in column 2 at x %d; 7 chips on %d lines; folded: %d fields reachable\n",
           f.x - g.x, lines, shown);
    gates_tree_destroy(t);
    return 0;
}
```

12.6 접근성

스핀 상자는 보이는 단위의 RangeValue(값 125에 배율 100이면 1.25, 작은·큰 변화는 칸과 쪽)가 있는 Spinner이고, 그 글상자는 스핀 상자의 이름을 가진 편집기이다. 슬라이더는 RangeValue가 있는 Slider이다. 화면 낭독기와 자동화는 어느 쪽 값이든 정할 수 있으며, 그 값은 범위 안으로 맞춰지고 사람의 변경처럼 알려진다. 그룹 상자는 제목을 이름으로 가진 Group이다. 접을 수 있는 그룹에는 ExpandCollapse가 있고, 그 제목의 초점은 그룹의 초점이다.

13 이미지와 네이티브 대화 상자

이 장의 차례

13.1 이미지 55
13.2 아이콘 55
13.3 네이티브 대화 상자 56

헤더: `gates/image.h`, `gates/draw.h`, `gates/window.h`.

13.1 이미지

트리는 이미지를 id로 보관한다. `gates_image_add_rgba(tree, w, h, pixels, stride, &id)`는 곧은 알파 RGBA 픽셀을 복사해 넣고, `gates_image_load_file`과 `gates_image_load_memory`는 플랫폼이 설치한 디코더로 PNG, JPEG, BMP, GIF, ICO, TIFF를 푼다(Windows 창에는 모두 있고, 창 없는 트리는 UNSUPPORTED로 답한다). `gates_image_remove`는 이미지를 풀어 준다. 그 이미지를 보여 주던 노드는 아무것도 보이지 않으며, id는 다시 쓰이지 않는다.

이미지의 기본 크기는 픽셀 크기를 논리 단위로 본 것이다. 그래서 32 x 16 그림은 32 x 16 단위를 차지하고 글자처럼 창과 함께 커진다. 150 %에서는 48 x 24 픽셀에 부드럽게(쌍선형) 그려진다. 큰 배율에서도 선명해야 하는 그림에는 더 큰 픽셀을 준다.

이미지는 그 큰 화소를 스스로 지닐 수 있다(0.10.0). `gates_image_add_variant_rgba(tree, id, w, h, rgba, stride)`는 같은 그림을 다른 화소 크기로 더한다. 예를 들어 16 단위 아이콘을 24 화소와 32 화소로 그린 것이다. `gates_image_load_variant_file` / `_memory`는 그것을 디코드해서 더한다. 기본 크기는 첫 화소 묶음의 크기 그대로다. 이미지를 그릴 때마다 렌더러는 채울 장치 화소를 덮는 가장 작은 묶음을, 없으면 가장 큰 묶음을 쓴다. 100 %에서는 16 화소 묶음, 150 %에서는 24, 200 %에서는 32다. 묶음은 그림의 모양을 지켜야 하고(높이가 w x 기본 높이 / 기본 너비와 한 화소 안), 앞서 더한 묶음과 너비가 같으면 그것을 바꾼다. `gates_image_variant_count`, `gates_image_variant_size`, `gates_image_pick_size`는 이미지가 무엇을 지녔고 어떤 크기에 어느 묶음을 쓸지 알려 준다.

`gates_image_create(tree, parent, id, &node)`는 이미지를 보여 준다. `gates_image_node_set_size`는 다른 크기를 주며, 그때 그림은 가로세로 비율을 지키고 가운데에 놓인다. 접근성 이름(`gates_node_set_access_name`)이 있는 이미지는 화면 낭독기에 Image이고, 이름이 없으면 장식이라 읽히지 않는다.

13.2 아이콘

아이콘은 16 x 16 단위로 그려진다(`GATES_ICON_SIZE`). `gates_button_set_icon`은 버튼 레이블 왼쪽에 아이콘을 둔다. 아이콘만 있고 글자가 없는 버튼은 아이콘 버튼이며 접근성 이름이 있어야 한다. `gates_command_set_icon`은 명령에 아이콘을 준다. 메뉴는 여백 칸에 그리고(체크된 명령은 대신 표시를 보인다), 도구 막대는 레이블 왼쪽에 그리거나 `gates_toolbar_set_icons_only`로 아이콘만 그린다. 그때 레이블은 툴팁과 접근성 이름에 남는다.

직접 그릴 때는 `gates_tree_image`가 `gates_draw_image(dl, rect, image)`에 쓸 이미지를 돌려준다. 그리기 목록은 그것을 빌려 쓰므로 목록을 그릴 때까지 이미지를 두어야 한다.

```
/* manual example (host): an image from pixels, an icon on a button and on a command.
 * expect: logo 32x16, drawn 32x16 at 192 dpi as 64x32 pixels; Save button 20 units wider with its icon */
#include <gates/gates.h>

#include <stdio.h>

int main(void) {
```

```
    gates_tree_t *t = nullptr;
    if (!gates_is_ok(gates_tree_create(&(gates_tree_desc_t){0}, &t))) return 1;
    gates_node_t root = gates_tree_root(t), logo, save;
    /* 32 x 16 pixels, a left-to-right fade from blue to transparent. */
    static gates_u8 px[16][32][4];
    for (int y = 0; y < 16; y++) {
        for (int x = 0; x < 32; x++) {
            px[y][x][0] = 0; px[y][x][1] = 90; px[y][x][2] = 200; px[y][x][3] = (gates_u8)(255 - x * 8);
        }
    }
    /* A 16 x 16 icon: a grey square. */
    static gates_u8 ic[16][16][4];
    for (int y = 0; y < 16; y++) {
        for (int x = 0; x < 16; x++) {
            ic[y][x][0] = ic[y][x][1] = ic[y][x][2] = 90; ic[y][x][3] = 255;
        }
    }
    gates_image_id_t pic = 0, icon = 0;
    if (!gates_is_ok(gates_image_add_rgba(t, 32, 16, &px[0][0][0], 0, &pic)) ||
        !gates_is_ok(gates_image_add_rgba(t, 16, 16, &ic[0][0][0], 0, &icon)) ||
        !gates_is_ok(gates_layout_set(t, root, GATES_LAYOUT_KIND_COLUMN)) ||
        !gates_is_ok(gates_image_create(t, root, pic, &logo)) ||
        !gates_is_ok(gates_node_set_access_name(t, logo, GATES_STR("Logo"))) ||   /* named: an Image */
        !gates_is_ok(gates_layout_set_child_align(t, logo, GATES_ALIGN_START_V)) ||
        !gates_is_ok(gates_button_create(t, root, GATES_STR("Save"), nullptr, nullptr, &save)) ||
        !gates_is_ok(gates_layout_set_child_align(t, save, GATES_ALIGN_START_V)) ||
        !gates_is_ok(gates_layout_run(t, (gates_size_t){ 200, 100 }, gates_text_backend_builtin()))) {
        return 1;
    }
    gates_i32 plain = gates_node_preferred_size(t, save).w;
    if (!gates_is_ok(gates_button_set_icon(t, save, icon)) ||
        !gates_is_ok(gates_layout_run(t, (gates_size_t){ 200, 100 }, gates_text_backend_builtin()))) {
        return 1;
    }
    /* Render at 192 dpi: the logo covers 64 x 32 device pixels. */
    gates_draw_list_t dl;
    static gates_u32 pixels[200 * 2 * 100 * 2];
    gates_pixels_t target = { .ptr = pixels, .w = 400, .h = 200, .stride_bytes = 1600 };
    if (!gates_is_ok(gates_draw_list_init(&dl, (gates_allocator_t){0}, 0)) ||
        !gates_is_ok(gates_paint_tree(t, &dl, gates_theme_light(), gates_text_backend_builtin())) ||
        !gates_is_ok(gates_render_soft_scaled(&dl, target, gates_text_backend_builtin(), 192))) {
        return 1;
    }
    gates_rect_t r = gates_node_layout_rect(t, logo), dev = gates_rect_px(r, 192);
    printf("logo %dx%d, drawn %dx%d at 192 dpi as %dx%d pixels; Save button %d units wider with its icon\n",
           gates_image_size(t, pic).w, gates_image_size(t, pic).h, r.w, r.h, dev.w, dev.h,
           gates_node_preferred_size(t, save).w - plain);
    gates_draw_list_deinit(&dl);
    gates_tree_destroy(t);
    return 0;
}
```

13.3 네이티브 대화 상자

파일, 폴더, 색을 고르는 일과 예/아니오를 묻는 일은 플랫폼에 맡기는 것이 가장 좋다. 사람들은 그 대화 상자를 알고, 그것은 플랫폼의 키보드 지원, 화면 낭독기 지원, 최근 위치를 함께 가져온다. Windows에서는 다음과 같다.

- `gates_window_open_file`, `gates_window_save_file`, `gates_window_choose_folder`는 `gates_file_dialog_t`(제목, `"Pictures¦*.png;*.jpg¦All files¦*.*"` 같은 필터, 시작 폴더, 저장할 때의 추천 이름)를 받아 고른 경로를

UTF-8로 버퍼에 적는다. 저장 대화 상자는 덮어쓰기 전에 묻는다. `gates_window_open_files`는 파일 여러 개를 고르게 한다. 버퍼에서 경로마다 NUL 바이트로 끝나고, `count`가 몇 개인지 알려 준다.

- `gates_window_choose_color`는 한 색에서 시작해 고른 색을 답한다.
- `gates_window_message`는 확인, 확인/취소, 예/아니오, 예/아니오/취소 버튼과 정보, 경고, 오류, 질문 아이콘으로 메시지를 보이고 답을 돌려준다(Escape와 닫기 상자는 취소로 답한다).

이것은 플랫폼의 모달 대화 상자이다. 호출은 사람이 답할 때 돌아오고, 창의 메뉴는 먼저 닫힌다. 취소는 오류가 아니다. 경로가 비어서 돌아오거나 `chosen`이 거짓이다. 명령이나 이벤트 처리기에서 부르고, 그리는 도중에는 부르지 않는다.